Universidad Carlos III de Madrid
Computación Distribuida

Práctica entregable: despliegue de WordPress en AWS

Nivel básico, nivel medio y nivel avanzado (opción A, Amazon RDS)

Jacobo Gutiérrez - 100472851
Munir Akri - 100522755

Curso 2026/2027

Nivel básico: Arquitectura segura en la VPC

El objetivo de este nivel es montar una VPC propia con subredes públicas y privadas, un servidor público que hace de puerta de entrada y un servidor privado con MariaDB al que solo se llega desde el público.

1. Crear la VPC con subredes públicas y privadas

1.1. Configurar la VPC

Con el Learner Lab arrancado y la consola en us-east-1 (N. Virginia), entramos en el servicio VPC y usamos el asistente “VPC y más”, que crea de una vez la VPC, las subredes, las tablas de rutas y el Internet Gateway. Como prefijo para los nombres ponemos proyecto y como bloque IPv4 10.0.0.0/16, que da algo más de 65.000 direcciones para toda la VPC.

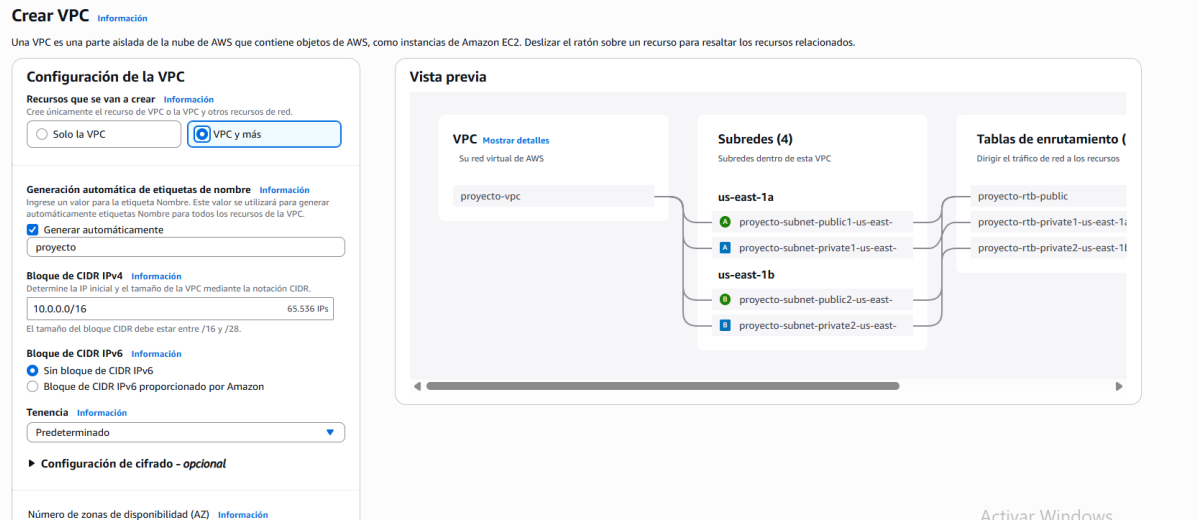


Figura 1. Asistente “VPC y más”: prefijo proyecto y bloque 10.0.0.0/16.

Elegimos dos zonas de disponibilidad, con dos subredes públicas y dos privadas, y dejamos sin NAT Gateway (tiene coste y aquí no lo necesitamos) y sin endpoints. En la vista previa de la derecha se ve lo que va a crear: cuatro subredes repartidas entre us-east-1a y us-east-1b, tres tablas de rutas y la conexión al Internet Gateway proyecto-igw.

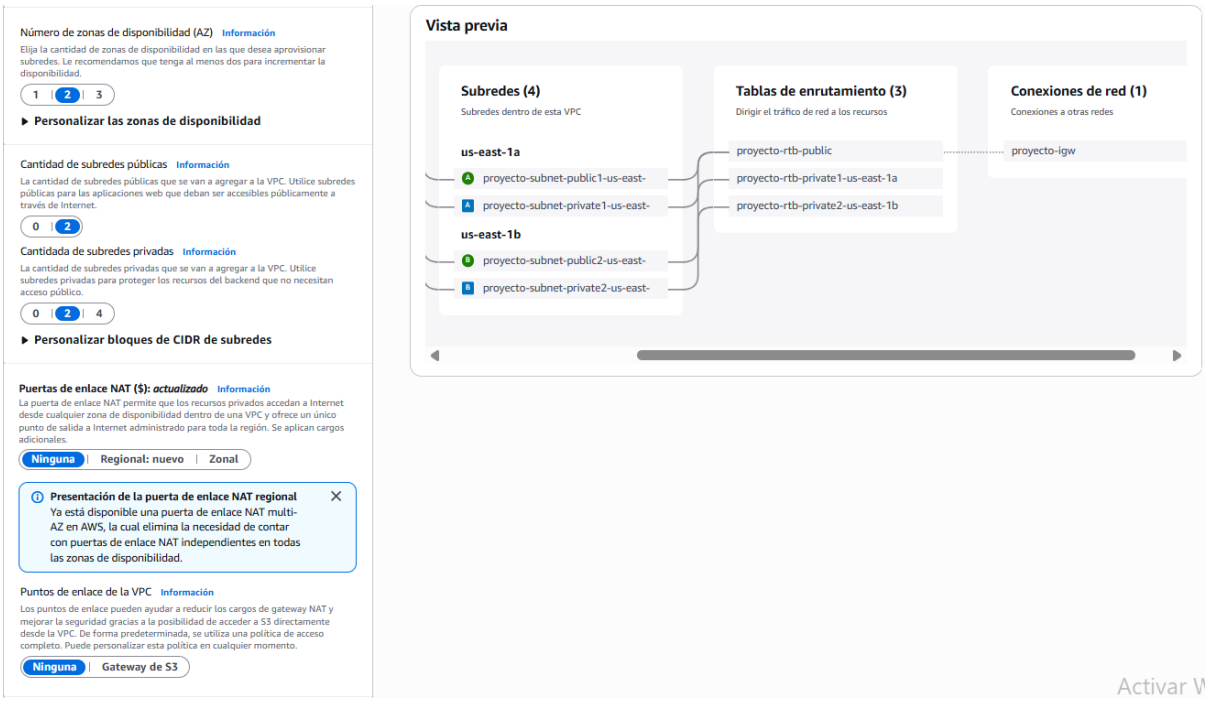


Figura 2. Dos zonas de disponibilidad, sin NAT Gateway, y vista previa de la red.

1.2. Verificar las subredes

En VPC → Subredes salen nuestras cuatro subredes mezcladas con las de la VPC por defecto (las de 172.31.x.x). En la segunda captura nos quedamos solo con las de proyecto-vpc para ver bien sus CIDR, que nos hacen falta después para elegir las IPs de las instancias.

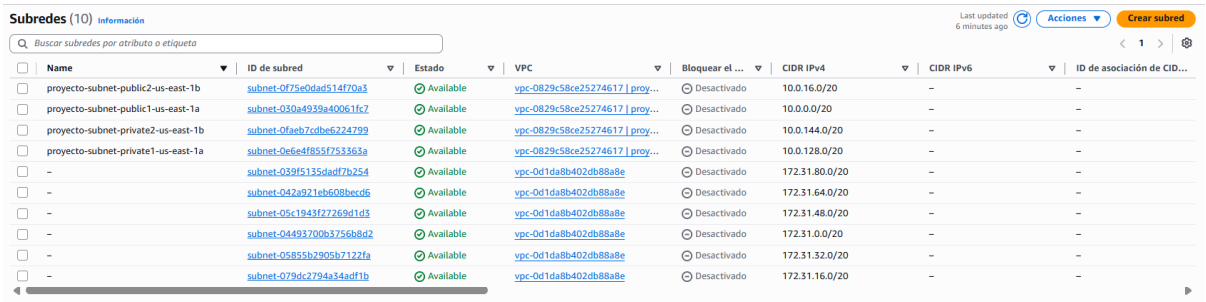


Figura 3. Subredes de la cuenta, junto a las de la VPC por defecto.

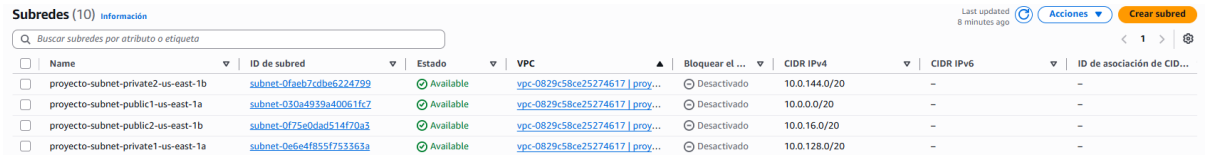


Figura 4. Las cuatro subredes de proyecto-vpc con sus CIDR.

Subred	Tipo	CIDR	AZ
proyecto-subnet-public1-us-east-1a	Pública	10.0.0.0/20	us-east-1a
proyecto-subnet-public2-us-east-1b	Pública	10.0.16.0/20	us-east-1b
proyecto-subnet-private1-us-east-1a	Privada	10.0.128.0/20	us-east-1a
proyecto-subnet-private2-us-east-1b	Privada	10.0.144.0/20	us-east-1b

En esta práctica solo usamos las dos de us-east-1a. Las de us-east-1b se quedan vacías, preparadas por si más adelante hiciera falta alta disponibilidad.

2. Crear los Security Groups

2.1. Security Group público

Creamos grupo-publico dentro de proyecto-vpc, que es el que llevará el servidor web. El SSH (22) solo lo permitimos desde 163.117.0.0/16, la red de la UC3M, para que la máquina no se pueda administrar desde cualquier sitio. HTTP (80) y HTTPS (443) quedan abiertos a 0.0.0.0/0, porque es una web y tiene que poder entrar cualquiera. Las reglas de salida las dejamos como vienen.

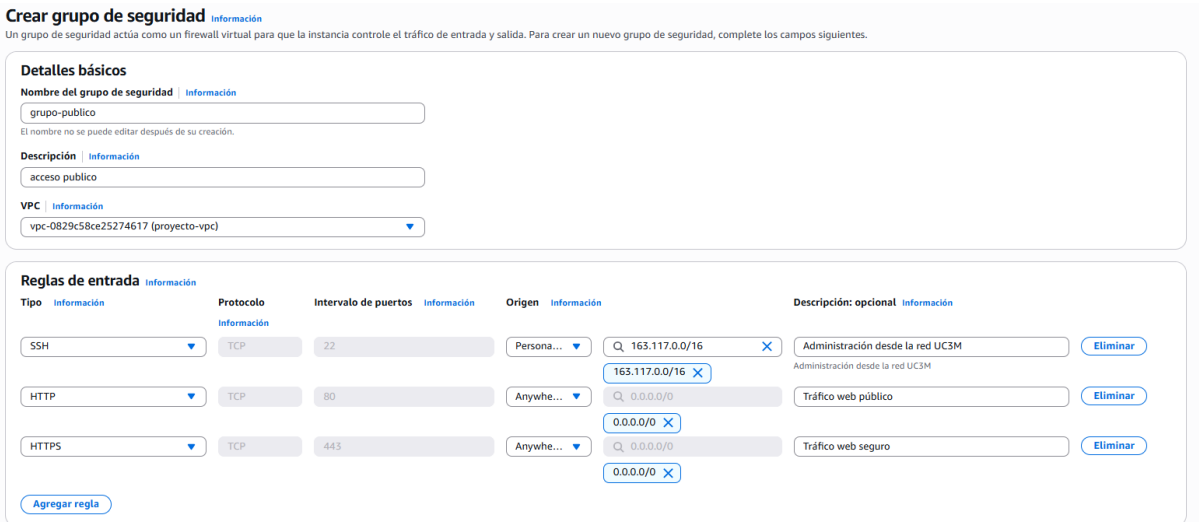


Figura 5. Creación de grupo-publico: SSH desde la UC3M, HTTP y HTTPS abiertos.

2.2. Security Group privado

grupo-privado es el de la base de datos. En las reglas de SSH (22) y MySQL/Aurora (3306) el origen no es una IP sino el propio grupo-publico. De esta forma solo entran las máquinas que tengan ese grupo, aunque cambien de IP, y alguien de fuera tendría que hacerse primero con la pública para poder llegar a la privada.

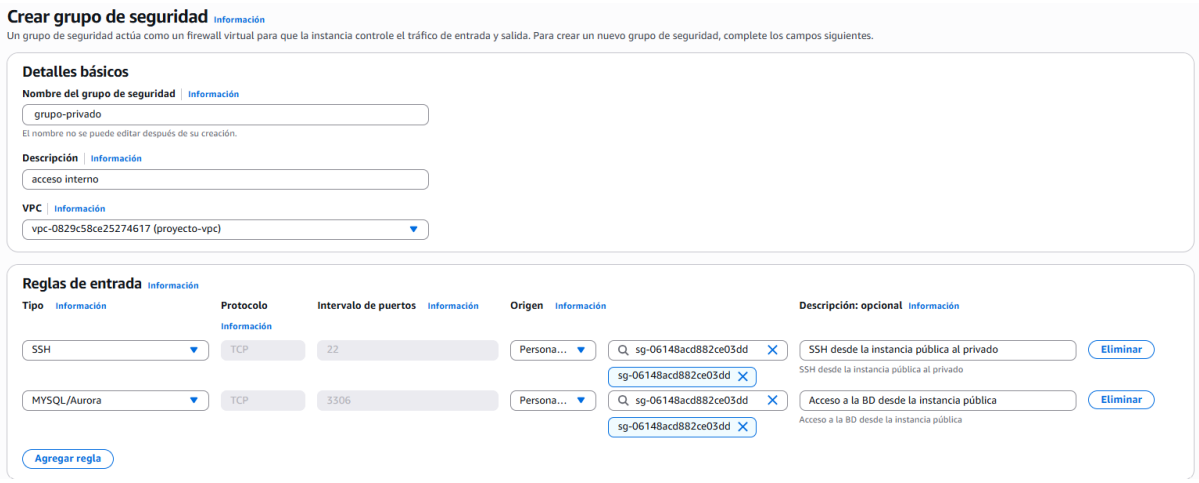


Figura 6. Creación de grupo-privado con grupo-publico como origen.

Una vez creado lo editamos otra vez para añadir una regla “Todo el tráfico” cuyo origen es el propio grupo-privado. No se podía poner al crearlo, porque el grupo aún no existía y no se puede elegir a sí mismo. Con ella se permite el tráfico entre todo lo que lleve este grupo, como la segunda interfaz de la instancia pública.

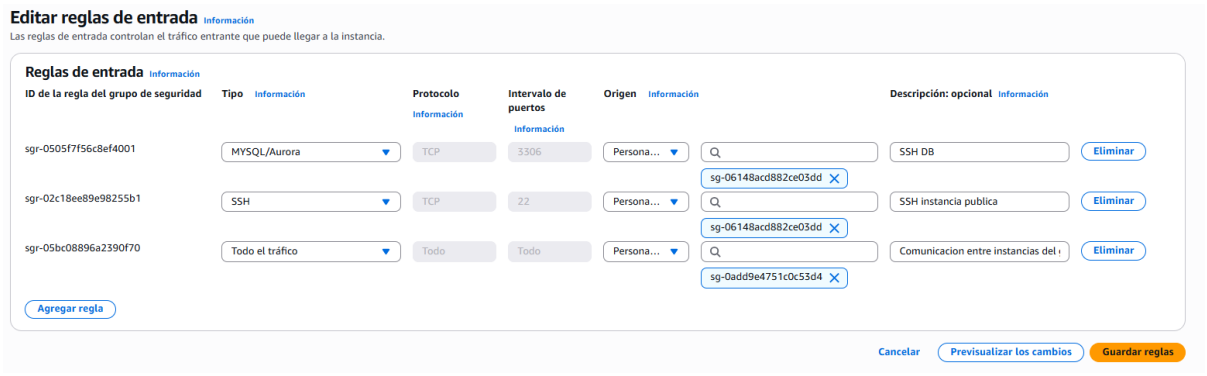


Figura 7. grupo-privado con la regla de todo el tráfico entre sus miembros.

3. Key Pair

No creamos uno nuevo: reutilizamos keypair-cdist, el que ya teníamos de prácticas anteriores (RSA, formato .pem). En EC2 → Pares de claves aparece junto a vockey, que es el que trae el laboratorio.

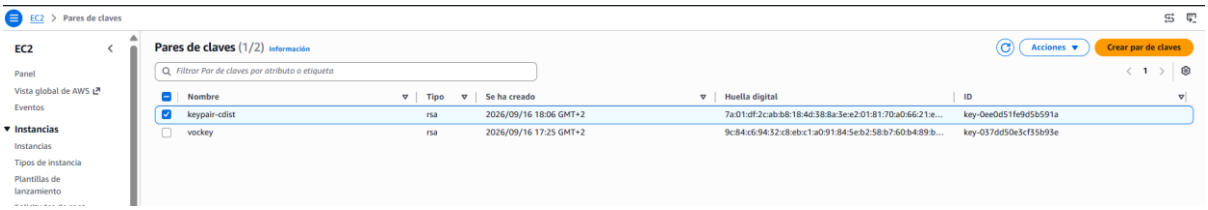


Figura 8. Par de claves keypair-cdist en la consola de EC2.

4. Crear las instancias EC2

La idea es la misma que si lo montáramos con máquinas físicas: la instancia pública tiene dos tarjetas de red, una conectada a la subred pública (la que da a internet) y otra a la privada, que es por donde habla con la base de datos. La privada solo tiene una, dentro de su subred.

4.1. Instancia pública: servidor-web-publico

Lanzamos una instancia llamada servidor-web-publico con Amazon Linux 2023, tipo t2.micro y el par de claves keypair-cdist.



Figura 9. Nombre de la instancia pública.

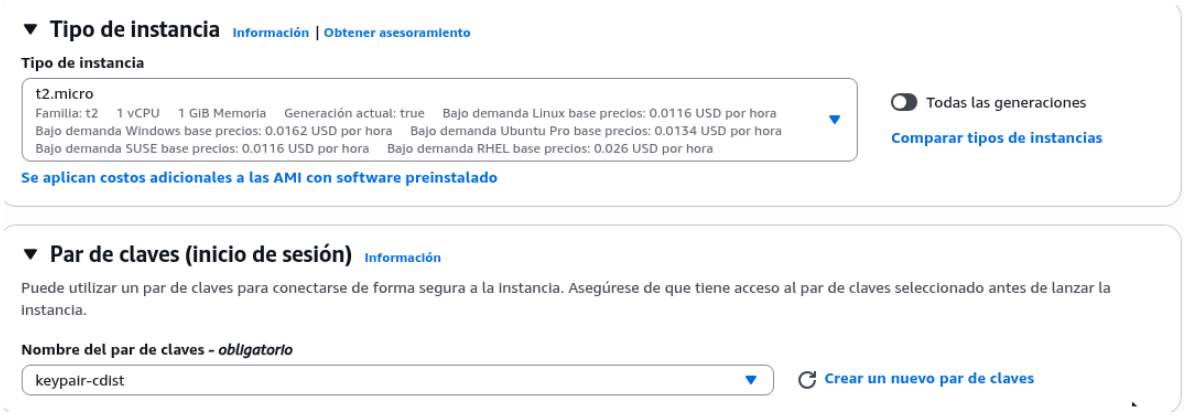


Figura 10. Tipo t2.micro y par de claves keypair-cdist.

En la configuración de red elegimos proyecto-vpc y la subred proyecto-subnet-public1-us-east-1a. Desactivamos la IP pública automática, porque más adelante le pondremos una Elastic IP, y dejamos vacío “Grupos de seguridad comunes”, ya que cada interfaz va a llevar su propio grupo.

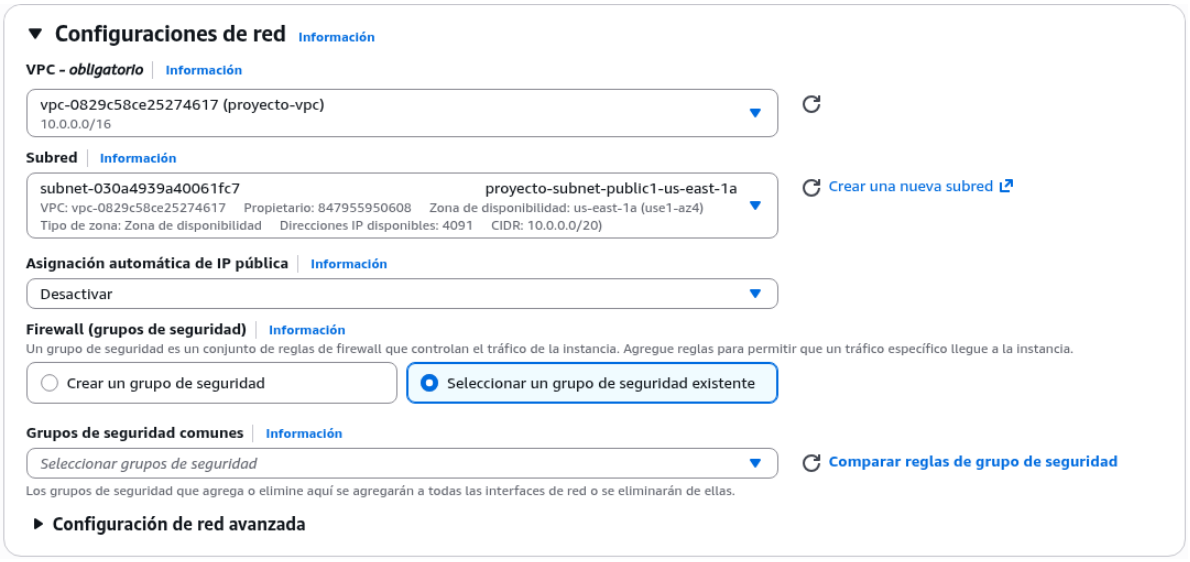


Figura 11. Red de la instancia pública: proyecto-vpc y subred pública de us-east-1a.

En “Configuración de red avanzada” definimos las dos interfaces. La primera va en la subred pública con grupo-publico y la IP 10.0.0.4, y la segunda en la subred privada con grupo-privado y la IP 10.0.128.4. Usamos la .4 porque AWS se reserva las cuatro primeras direcciones de cada subred.

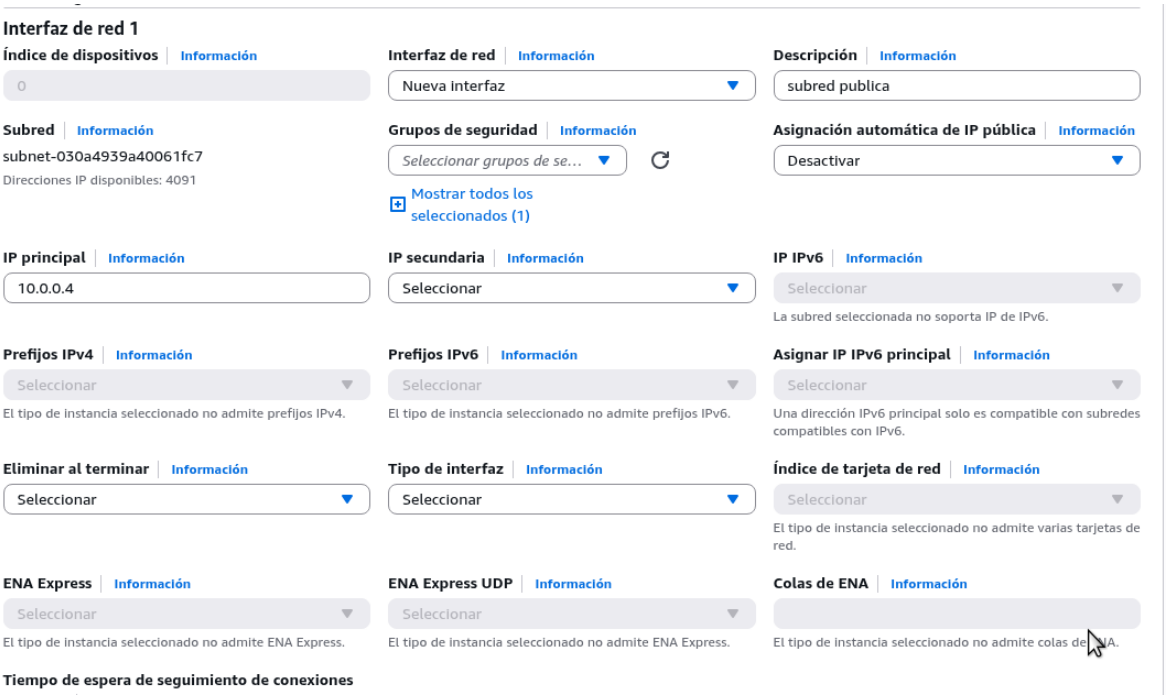


Figura 12. Primera interfaz: subred pública, IP 10.0.0.4.

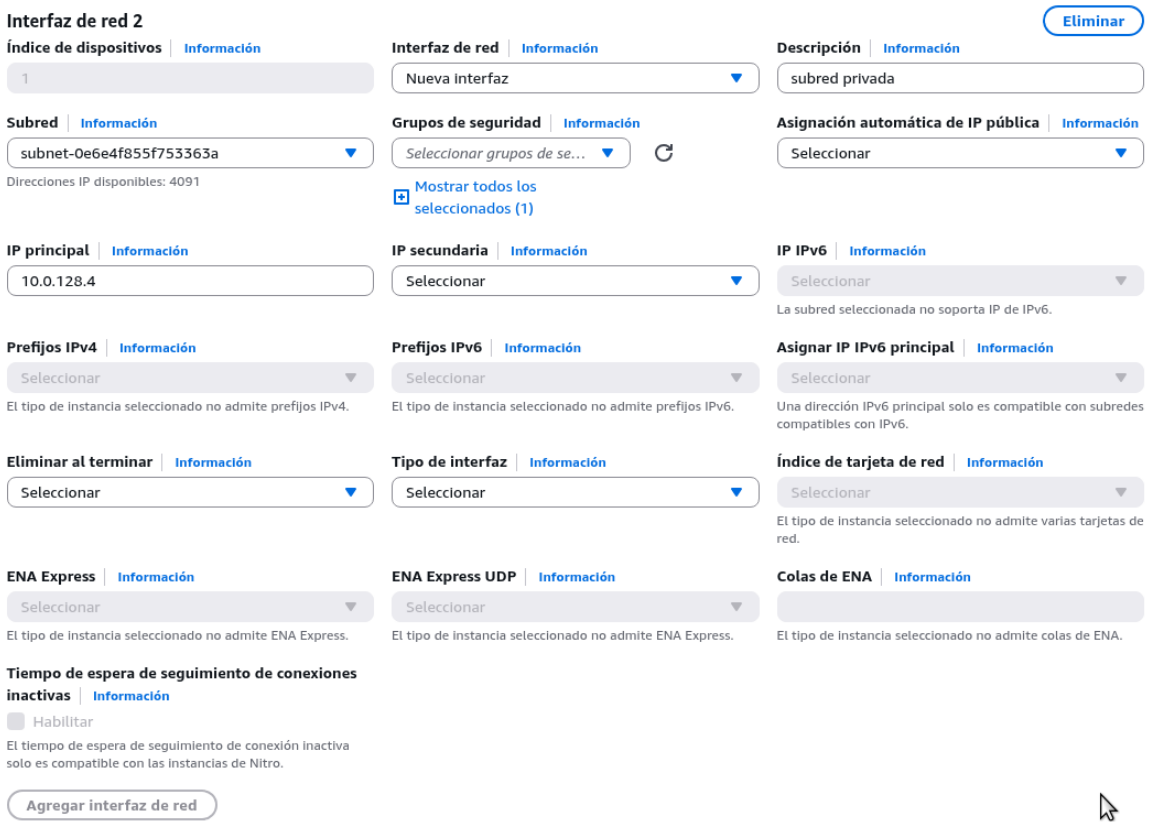


Figura 13. Segunda interfaz: subred privada, IP 10.0.128.4.

Por último, en “Detalles avanzados” seleccionamos LabInstanceProfile como perfil de IAM y lanzamos la instancia.

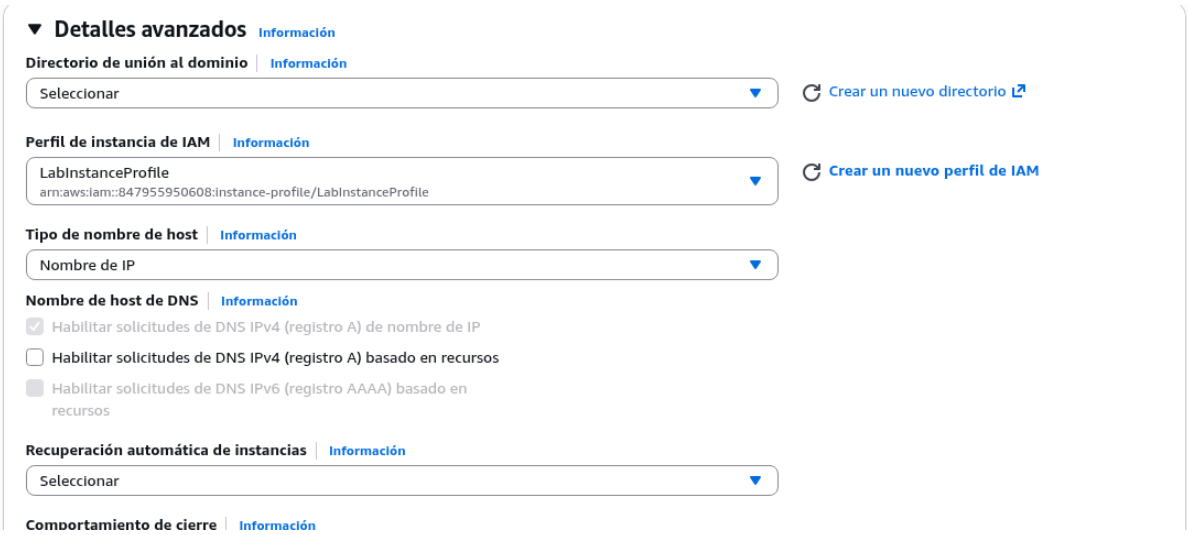


Figura 14. Perfil de IAM LabInstanceProfile en los detalles avanzados.

4.2. Instancia privada: servidor-privado

La segunda instancia se crea igual, con el mismo tipo y el mismo par de claves, pero con el nombre servidor-privado.

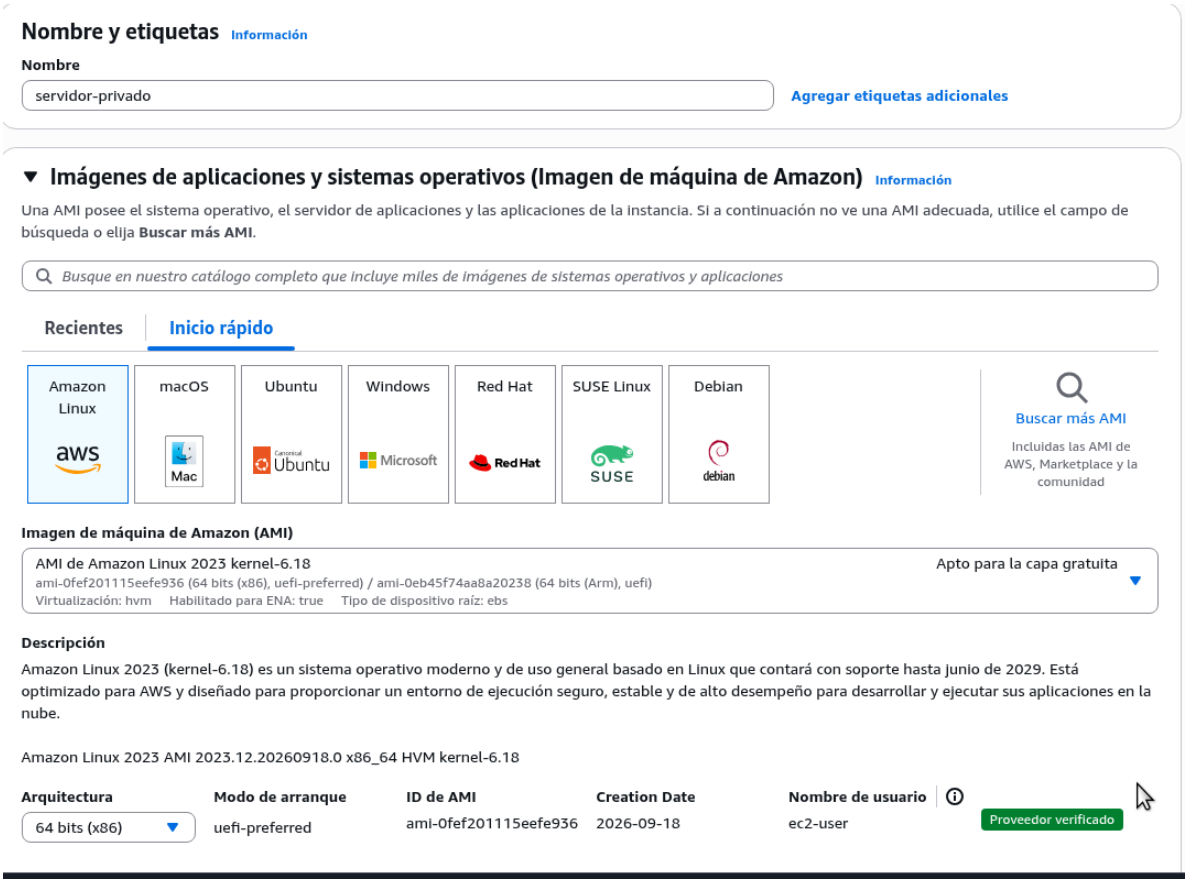


Figura 15. Nombre y AMI Amazon Linux 2023 de la instancia privada.

En red elegimos la subred proyecto-subnet-private1-us-east-1a, también con la IP pública desactivada. Esta vez solo hay una interfaz, con grupo-privado y la IP 10.0.128.5.

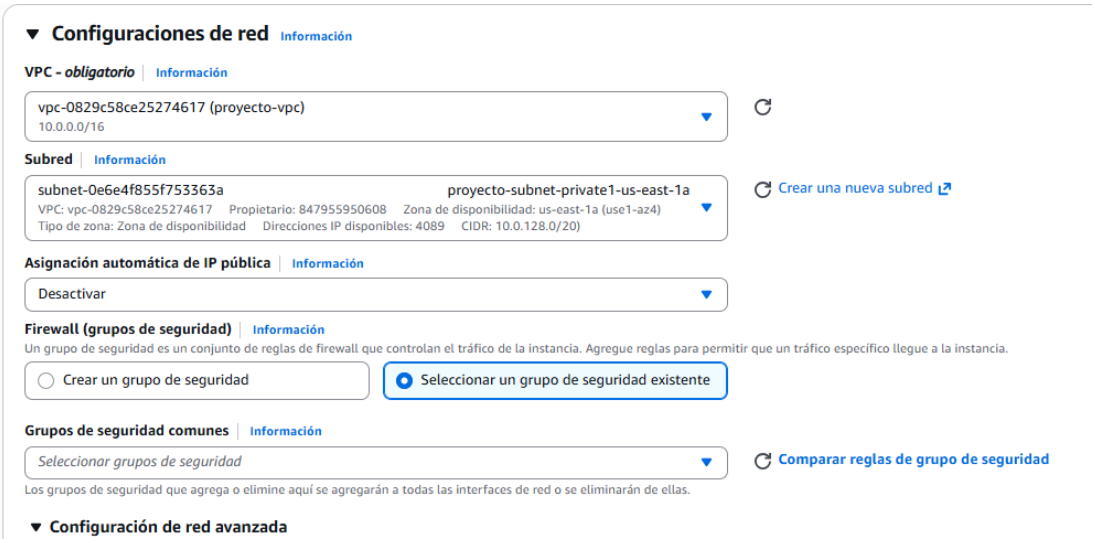


Figura 16. Red de la instancia privada: subred privada de us-east-1a.

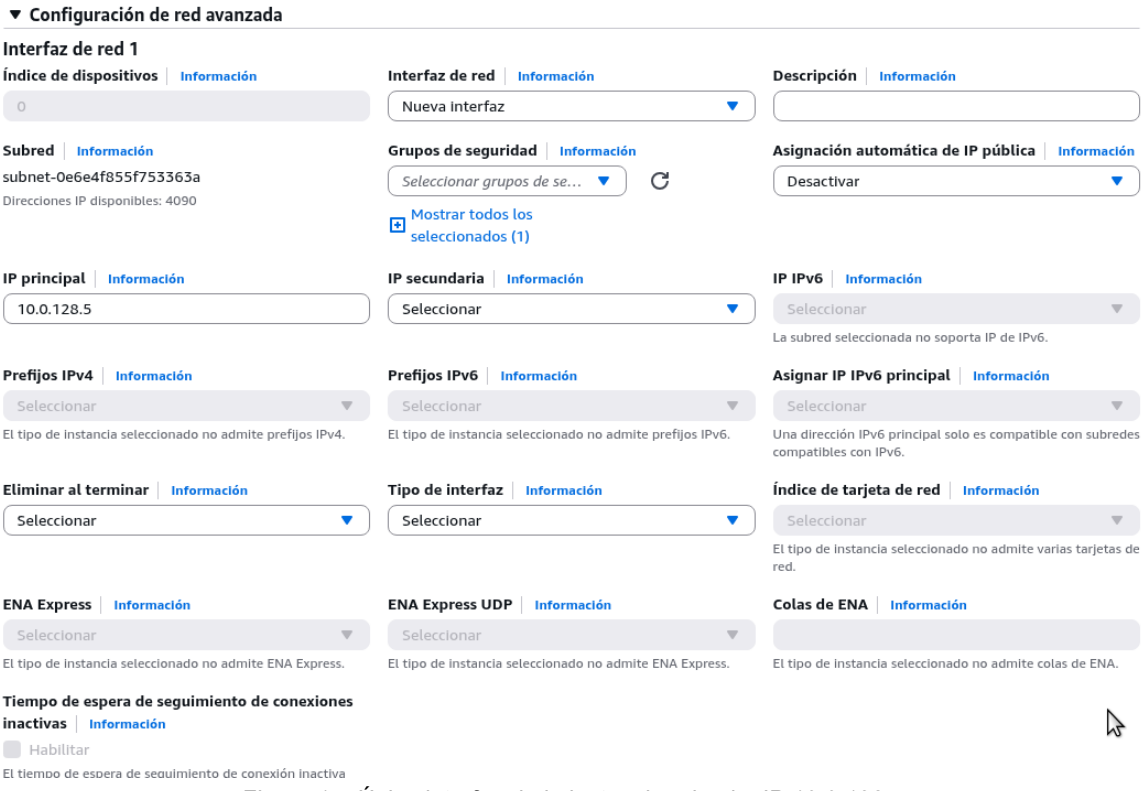


Figura 17. Única interfaz de la instancia privada, IP 10.0.128.5.

Si seleccionamos la instancia en la consola, en sus interfaces de red solo aparece la 10.0.128.5, sin IP ni DNS públicos, así que desde internet no hay forma de llegar a ella.

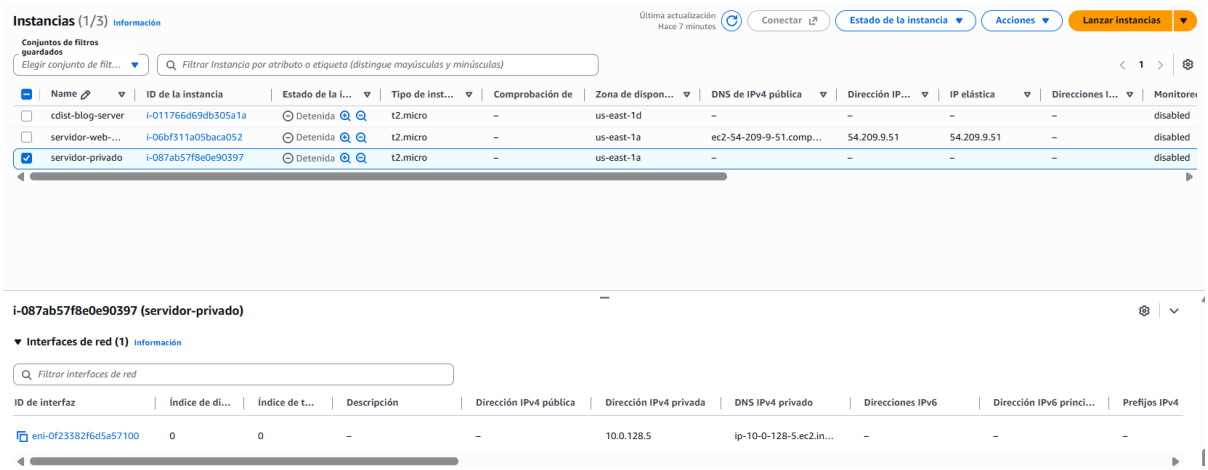


Figura 18. servidor-privado sin IP ni DNS públicos.

4.3. Verificar las instancias

En la lista de instancias aparecen las dos en ejecución en us-east-1a, todavía inicializándose y sin IP pública (cdist-blog-server es la de cloud-4 y está parada). Seleccionando la pública, en la pestaña “Redes” se ven sus dos IPs privadas, 10.0.0.4 y 10.0.128.4.

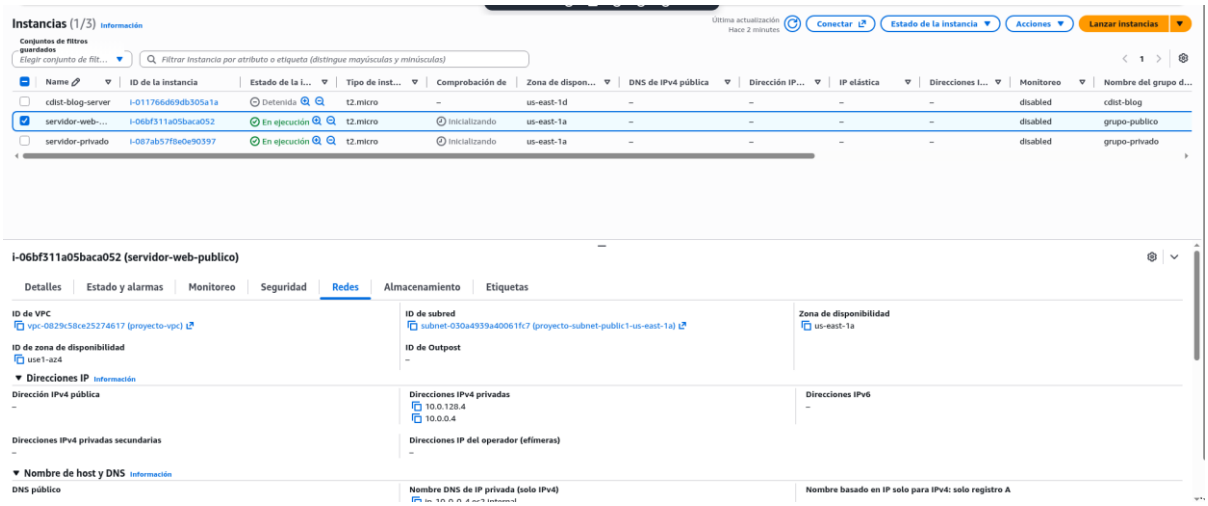


Figura 19. Las dos instancias arrancando y las IP privadas de la pública.

En el apartado de interfaces de red de la pública salen las dos ENI, cada una con su descripción (subred pública y subred privada) y su IP.

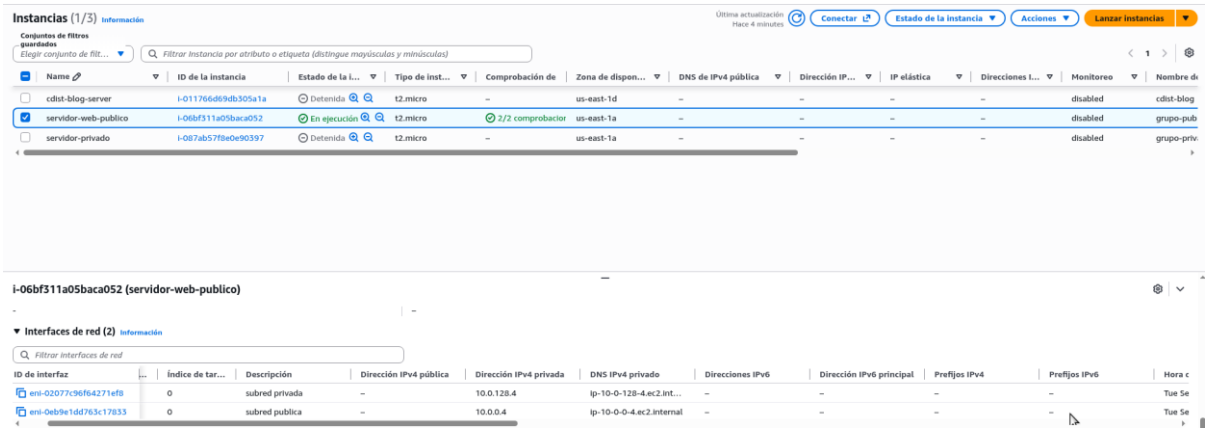


Figura 20. Interfaces de red de servidor-web-publico.

5. Elastic IP y conexión SSH al servidor público

5.1. Asignar la Elastic IP

Reservamos una Elastic IP en us-east-1, que nos da la 54.209.9.51. A diferencia de la IP automática, esta no cambia aunque paremos y arranquemos la instancia. Como la pública tiene dos interfaces, no la asociamos a la instancia sino directamente a la interfaz que tiene la 10.0.0.4, para que no acabe en la privada. En el resumen se ve la IP privada a la que ha quedado asociada y el DNS público que genera AWS.



Figura 21. Elastic IP 54.209.9.51 asociada a la interfaz de 10.0.0.4.

En la lista de instancias, servidor-web-publico ya aparece con su DNS e IP pública.



Figura 22. servidor-web-publico ya con IP pública.

5.2. Conectar por SSH

Desde el terminal del laboratorio nos conectamos con la clave y la Elastic IP. Al ser la primera vez, SSH pide aceptar la huella del servidor.

```
ssh -i ~/keypair-cdist.pem ec2-user@54.209.9.51
```

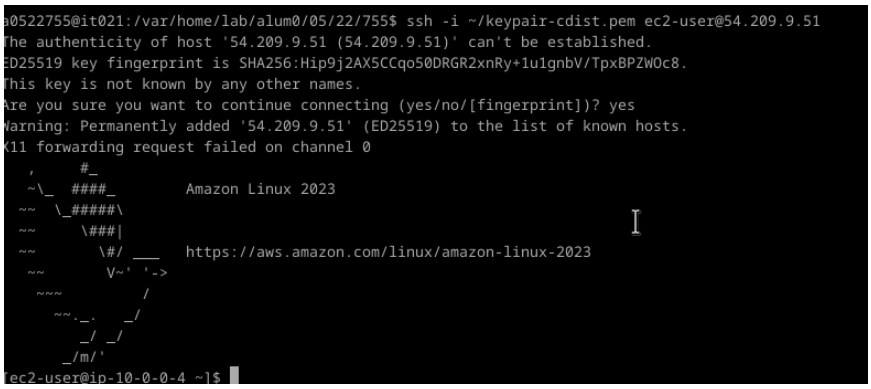


Figura 23. Primera conexión SSH al servidor público.

Ya dentro, hostname -I devuelve las dos IPs privadas de la máquina y con ip addr vemos a qué interfaz corresponde cada una: enX0 tiene la 10.0.0.4/20 y enX1 la 10.0.128.4/20. La IP pública no aparece en ningún sitio porque la traduce AWS desde fuera; la máquina ni se entera.

```
hostname -I
ip addr
```

```
[ec2-user@ip-10-0-0-4 ~]$ hostname -I
10.0.0.4 10.0.128.4
[ec2-user@ip-10-0-0-4 ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enX0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc fq_codel state UP group default qlen 1000
    link/ether 0a:ff:c3:d8:32:a3 brd ff:ff:ff:ff:ff:ff
    altname eni-0eb9e1dd763c17833
    altname device-number-0.0
    inet 10.0.0.4/20 metric 512 brd 10.0.15.255 scope global dynamic enX0
       valid_lft 2847sec preferred_lft 2847sec
    inet6 fe80::8ff:c3ff:fed8:32a3/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: enX1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc fq_codel state UP group default qlen 1000
    link/ether 0a:ff:e5:54:2e:9f brd ff:ff:ff:ff:ff:ff
    altname eni-02077c96f64271ef8
    altname device-number-1.0
    inet 10.0.128.4/20 metric 513 brd 10.0.143.255 scope global dynamic enX1
       valid_lft 2847sec preferred_lft 2847sec
    inet6 fe80::8ff:e5ff:fe54:2e9f/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figura 24. hostname -I e ip addr: enX0 con 10.0.0.4 y enX1 con 10.0.128.4.

6. Comprobar el routing entre subredes

6.1. Tabla de rutas

Con ip route vemos la tabla de rutas de la pública. Hay una ruta por defecto por cada interfaz (la de enX0 tiene menor métrica, así que es la que se usa) y, lo que nos interesa, una ruta directa a 10.0.128.0/20 por enX1. Es decir, la máquina ya sabe que para llegar a la subred privada tiene que salir por su segunda tarjeta.

```
[ec2-user@ip-10-0-0-4 ~]$ ip route
default via 10.0.0.1 dev enX0 proto dhcp src 10.0.0.4 metric 512
default via 10.0.128.1 dev enX1 proto dhcp src 10.0.128.4 metric 513
10.0.0.0/20 dev enX0 proto kernel scope link src 10.0.0.4 metric 512
10.0.0.1 dev enX0 proto dhcp scope link src 10.0.0.4 metric 512
10.0.0.2 dev enX0 proto dhcp scope link src 10.0.0.4 metric 512
10.0.0.2 via 10.0.128.1 dev enX1 proto dhcp src 10.0.128.4 metric 513
10.0.128.0/20 dev enX1 proto kernel scope link src 10.0.128.4 metric 513
10.0.128.1 dev enX1 proto dhcp scope link src 10.0.128.4 metric 513
[ec2-user@ip-10-0-0-4 ~]$
```

Figura 25. Tabla de rutas del servidor público.

6.2. Prueba de conectividad

Para comprobarlo hacemos ping a la 10.0.128.5. Responde a los 20 paquetes, casi todos por debajo del milisegundo, así que no hace falta añadir ninguna ruta a mano con ip route add.

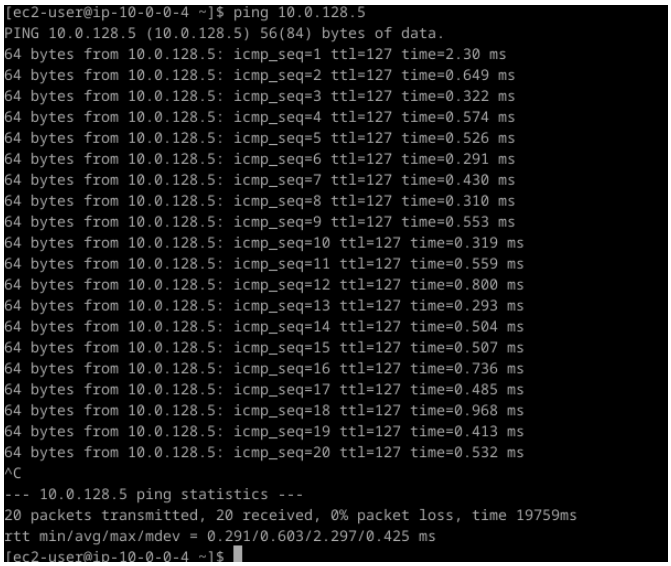


Figura 26. Ping del servidor público al privado.

7. Acceder al servidor privado (bastion host)

7.1. Copiar la clave a la pública

La privada no tiene IP pública, así que solo se puede entrar a ella pasando por la pública. Para eso la pública necesita la clave privada, y se la copiamos con scp desde el laboratorio (las dos instancias usan el mismo par de claves).

```
scp -i ~/keypair-cdist.pem ~/keypair-cdist.pem \
    ec2-user@54.209.9.51:/home/ec2-user/keypair-cdist.pem
```

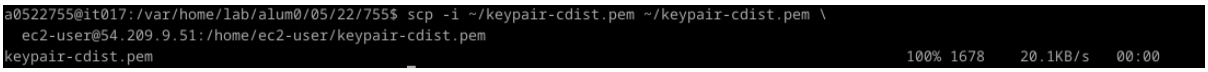


Figura 27. Copia de la clave al servidor público con scp.

7.2. Conectar al servidor privado

En la pública le damos permisos 400 a la clave, porque SSH no acepta una clave que puedan leer otros usuarios, y nos conectamos a la 10.0.128.5. El prompt pasa a ip-10-0-128-5, así que ya estamos en el servidor privado.

```
chmod 400 keypair-cdist.pem
ssh -i keypair-cdist.pem ec2-user@10.0.128.5
```

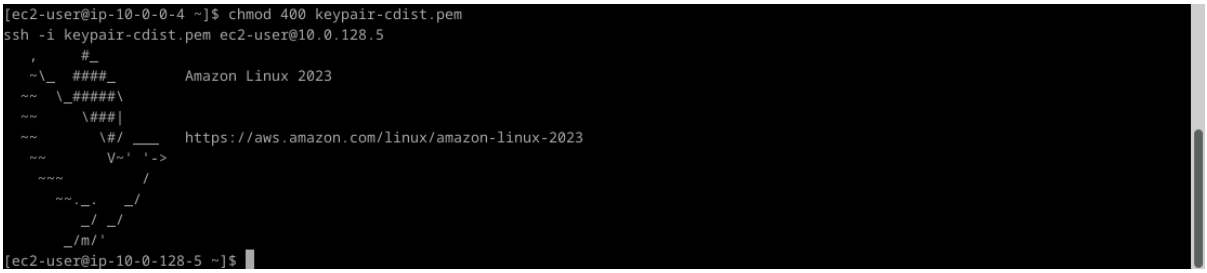


Figura 28. Conexión SSH al servidor privado a través del público.

Dentro, hostname -I solo devuelve la 10.0.128.5. Si probamos a hacer ping a www.google.com el nombre se resuelve, porque el DNS es el interno de la VPC, pero no vuelve ningún paquete: la subred privada no tiene salida a internet, que es justo lo que queremos.

```
[ec2-user@ip-10-0-128-5 ~]$ hostname -I
10.0.128.5
```

Figura 29. IP del servidor privado.

```
[ec2-user@ip-10-0-128-5 ~]$ ping www.google.com
PING www.google.com (142.251.155.119) 56(84) bytes of data.
^C
--- www.google.com ping statistics ---
11 packets transmitted, 0 received, 100% packet loss, time 10364ms
```

Figura 30. Sin salida a internet desde el servidor privado.

8. Dar salida temporal a internet al servidor privado

El problema es que sin internet tampoco podemos instalar MariaDB. Como no queremos pagar un NAT Gateway, le damos salida temporalmente y la quitamos en cuanto terminemos.

8.1. Segunda Elastic IP

Reservamos otra Elastic IP, la 52.55.192.195, y esta vez la asociamos eligiendo “Instancia”: la privada solo tiene una interfaz, así que no hay duda de dónde va a acabar.

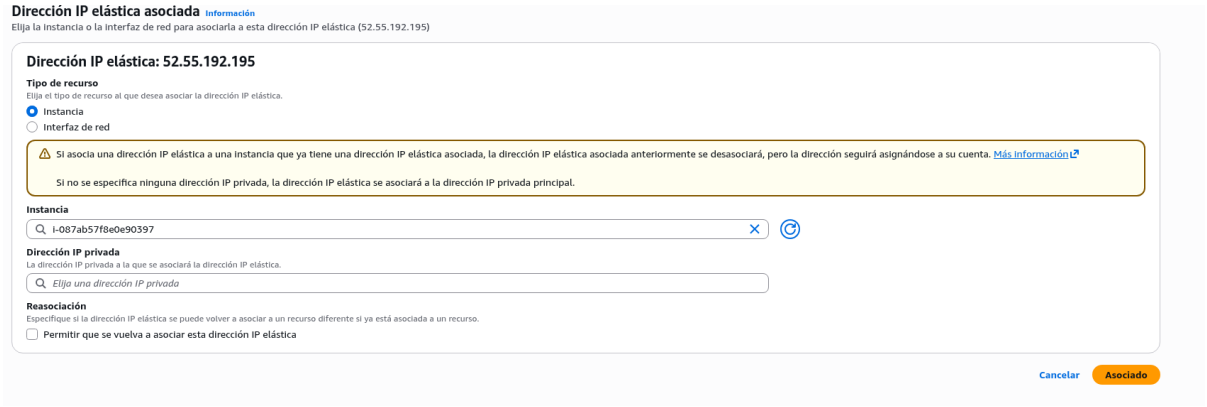


Figura 31. Asociación de la segunda Elastic IP al servidor privado.

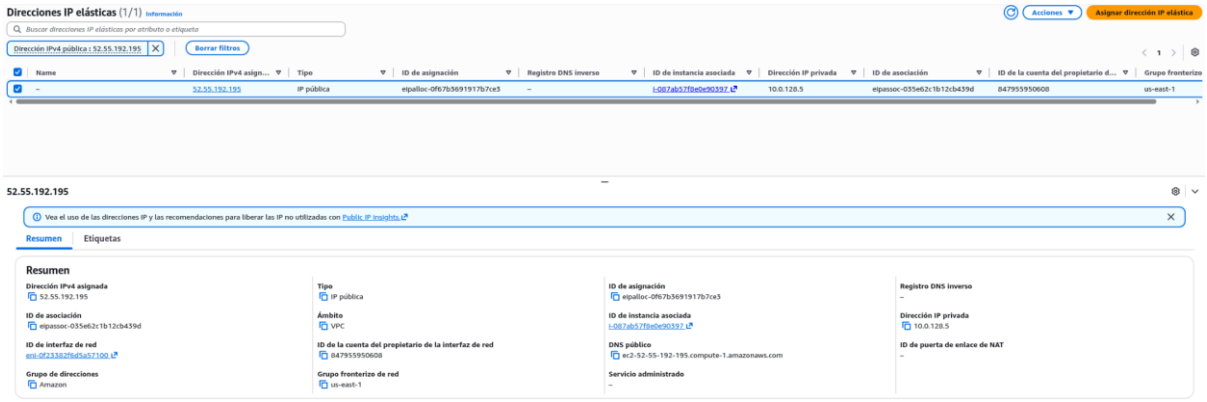


Figura 32. Elastic IP 52.55.192.195 asociada al servidor privado.

En la lista de instancias, servidor-privado aparece ahora también con IP pública.

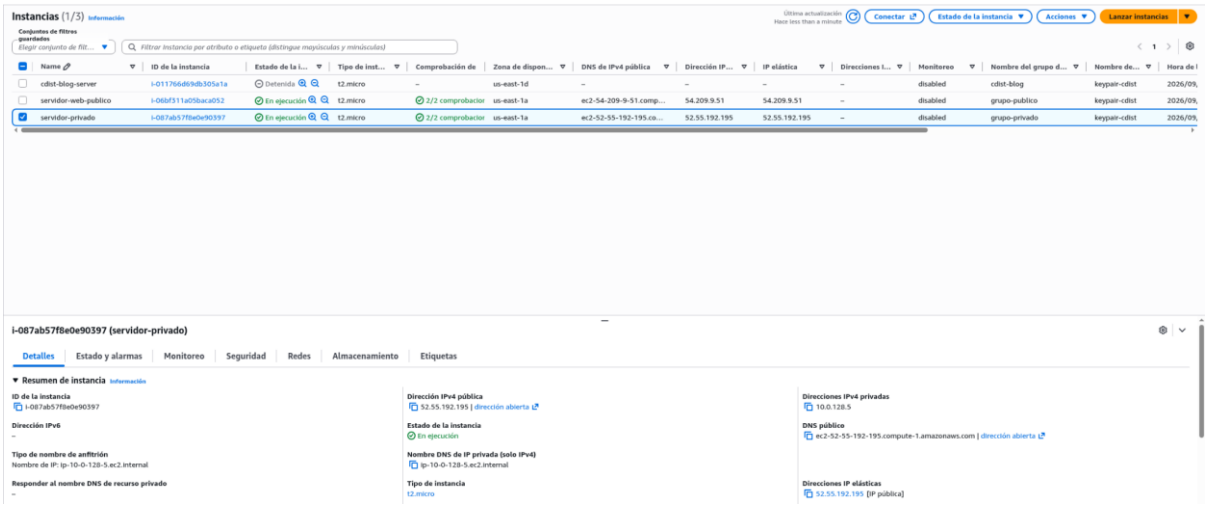


Figura 33. servidor-privado con IP pública temporal.

8.2. Ruta hacia internet

Con la IP sola no basta, porque la tabla de rutas de la subred privada solo conoce la red local 10.0.0.0/16. Editamos proyecto-rtb-private1-us-east-1a y añadimos la ruta 0.0.0.0/0 hacia el Internet Gateway. Esto se cambia en la tabla de rutas de AWS, no en el sistema operativo de la instancia.

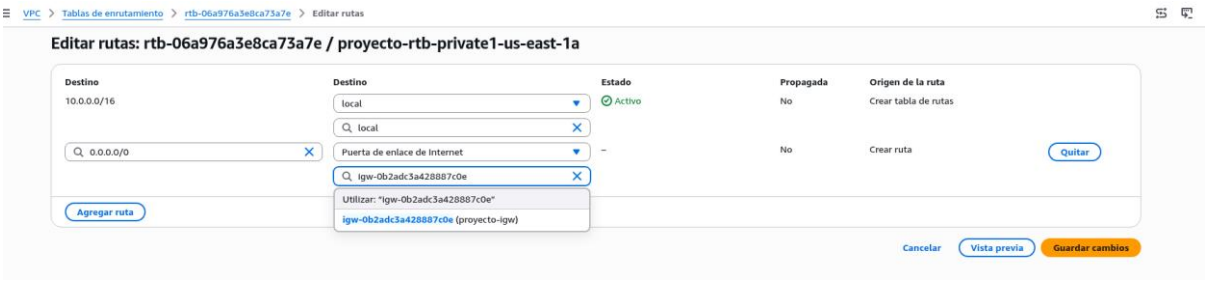


Figura 34. Ruta 0.0.0.0/0 hacia el Internet Gateway en la tabla privada.

Ahora el ping a www.google.com desde la privada sí responde. Mientras tenga la Elastic IP y esta ruta, la subred privada funciona en la práctica como una pública, por eso lo deshacemos al terminar con MariaDB (apartado 11).

```
[ec2-user@ip-10-0-128-5 ~]$ ping www.google.com
PING www.google.com (142.251.153.119) 56(84) bytes of data.
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=1 ttl=116 time=1.50 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=2 ttl=116 time=1.51 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=3 ttl=116 time=1.46 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=4 ttl=116 time=1.51 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=5 ttl=116 time=1.48 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=6 ttl=116 time=1.53 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=7 ttl=116 time=1.64 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=8 ttl=116 time=1.60 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=9 ttl=116 time=1.45 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=10 ttl=116 time=1.77 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=11 ttl=116 time=1.46 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=12 ttl=116 time=1.95 ms
64 bytes from 142.251.153.119 (142.251.153.119): icmp_seq=13 ttl=116 time=1.78 ms
^C
--- www.google.com ping statistics ---
13 packets transmitted, 13 received, 0% packet loss, time 12022ms
rtt min/avg/max/mdev = 1.454/1.587/1.945/0.148 ms
```

Figura 35. El servidor privado ya llega a internet.

9. Instalar MariaDB en el servidor privado

9.1. Actualizar e instalar

Desde la privada actualizamos el sistema e instalamos mariadb105-server, que se trae también el cliente, las librerías comunes y algunas dependencias de Perl. El dnf update no tenía nada que hacer porque la AMI era reciente.

```
sudo dnf update -y
sudo dnf install -y mariadb105-server
```

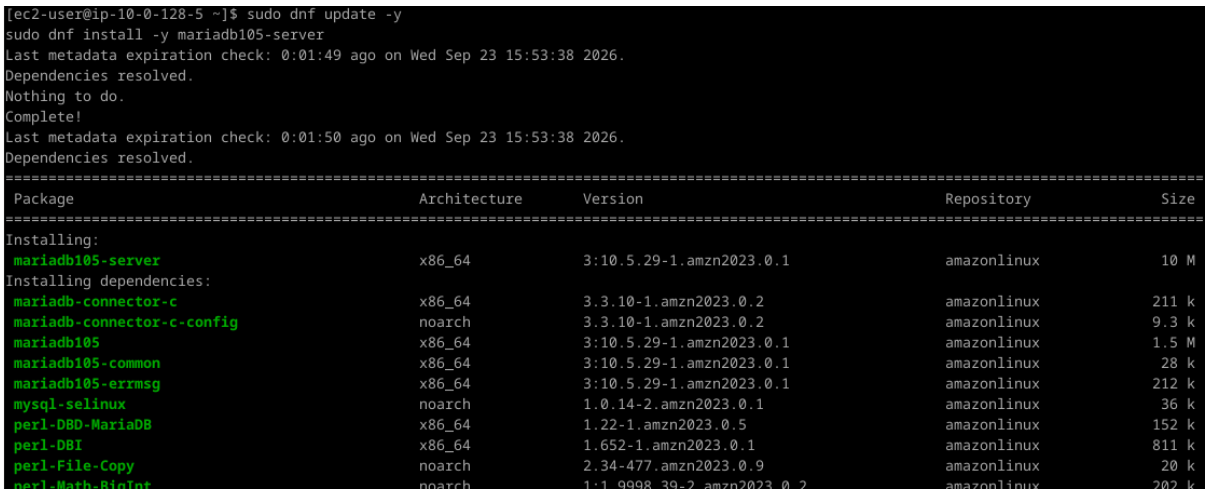


Figura 36. Instalación de mariadb105-server.

9.2. Arrancar y asegurar

Arrancamos el servicio, lo habilitamos para que se inicie solo y lanzamos mysql_secure_installation. Ponemos contraseña a root, quitamos los usuarios anónimos y la base de datos test que viene por defecto, y recargamos los privilegios. A la pregunta de deshabilitar el login remoto de root respondemos “n”, como indica el guion.

```
sudo systemctl start mariadb
sudo systemctl enable mariadb
sudo mysql_secure_installation
```

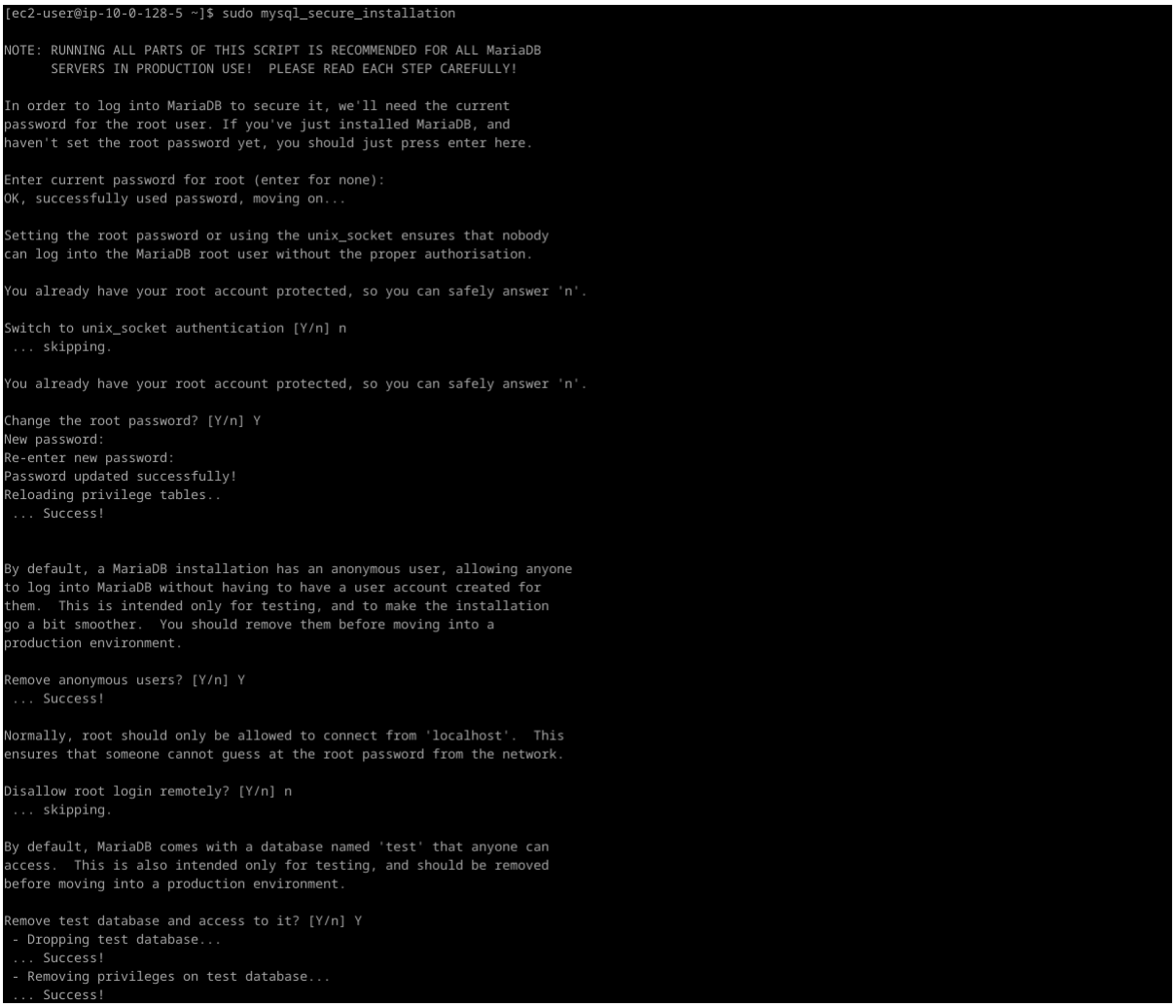


Figura 37. mysql_secure_installation.

9.3. Usuario y base de datos de prueba

Entramos como root, creamos el usuario publico y una base de datos test, y le damos todos los permisos sobre ella. Usamos % como host para que pueda conectarse desde cualquier IP. Para esta prueba nos vale; en el nivel medio el usuario que usa WordPress ya lo restringimos a la IP del servidor web.

```
CREATE USER 'publico'@'%' IDENTIFIED BY 'TU_PASSWORD_PUBLICO';
CREATE DATABASE `test`;
GRANT ALL PRIVILEGES ON `test`.* TO 'publico'@'%';
FLUSH PRIVILEGES;
```

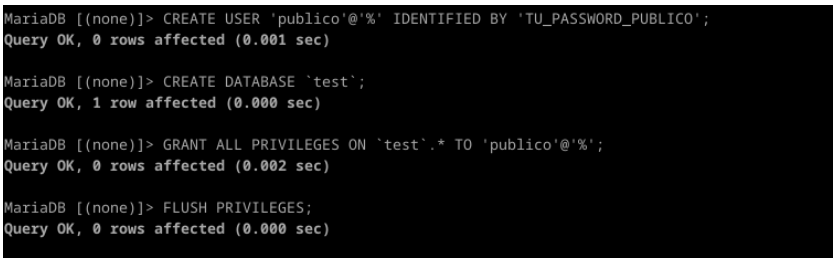


Figura 38. Usuario publico y base de datos test.

10. Probar la conexión desde el servidor público

10.1. Instalar el cliente de MariaDB

Salimos de la privada con exit y, de vuelta en la pública, instalamos solo el cliente (mariadb105). Aquí no va a haber ninguna base de datos, solo necesitamos poder conectarnos a la remota.

```
sudo dnf install -y mariadb105
```

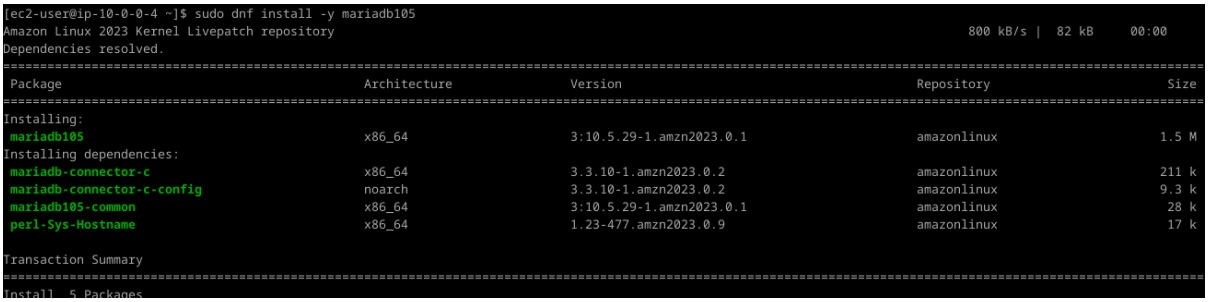


Figura 39. Cliente de MariaDB en el servidor público.

10.2. Conectar a la base de datos del privado

Nos conectamos al MariaDB de la privada indicando su IP con -h y el usuario publico. Entra a la primera y nos muestra el monitor de MariaDB 10.5.29.

```
mysql -h 10.0.128.5 -u publico -p
```



Figura 40. Conexión desde el público al MariaDB del privado.

Con SHOW DATABASES aparece test, entramos con USE test y SHOW TABLES sale vacío, lo normal porque todavía no hemos creado ninguna tabla. Con esto queda comprobado que la pública llega a la base de datos de la privada por la red interna de la VPC.

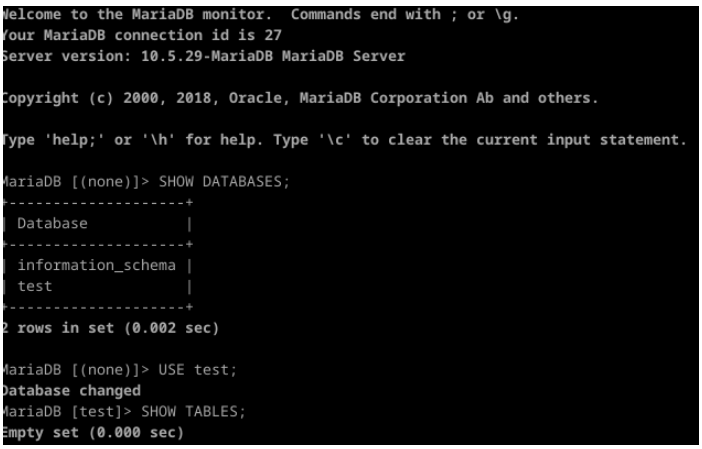


Figura 41. Bases de datos visibles para el usuario publico.

11. Retirar el acceso público al servidor privado

11.1. Quitar la Elastic IP

Con MariaDB instalado y probado, quitamos el acceso temporal. En VPC → IPs elásticas desasociamos la 52.55.192.195 de la privada y después la liberamos, para no gastar créditos con una IP que no se usa. En la segunda captura solo queda la 54.209.9.51, que es la de la pública.

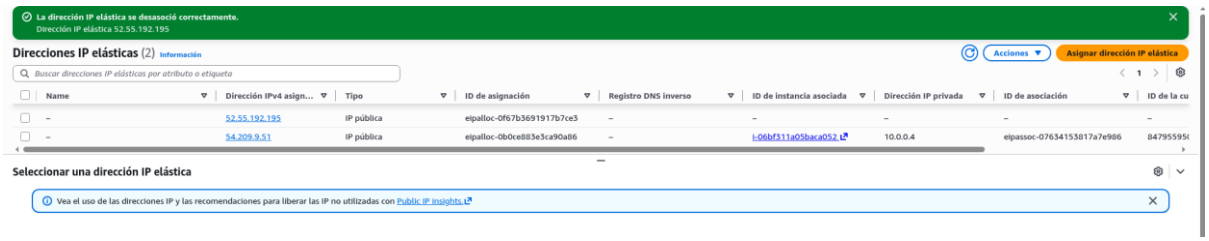


Figura 42. Desasociación de la Elastic IP del servidor privado.

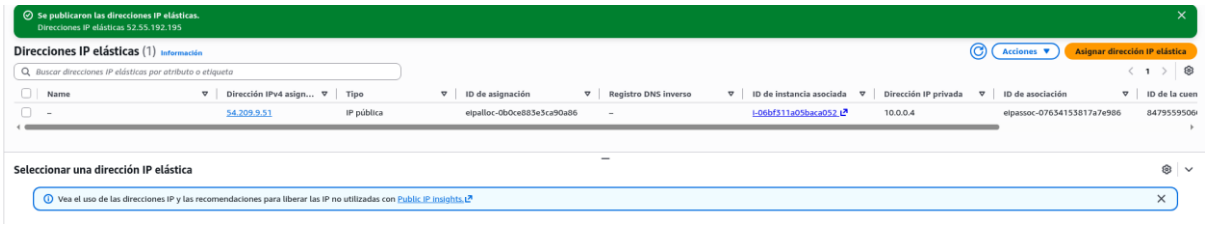


Figura 43. Elastic IP liberada; solo queda la del servidor público.

11.2. Quitar la ruta a internet

En proyecto-rtb-private1-us-east-1a borramos la ruta 0.0.0.0/0 hacia el Internet Gateway y guardamos. La tabla vuelve a tener solo la ruta local 10.0.0.0/16.

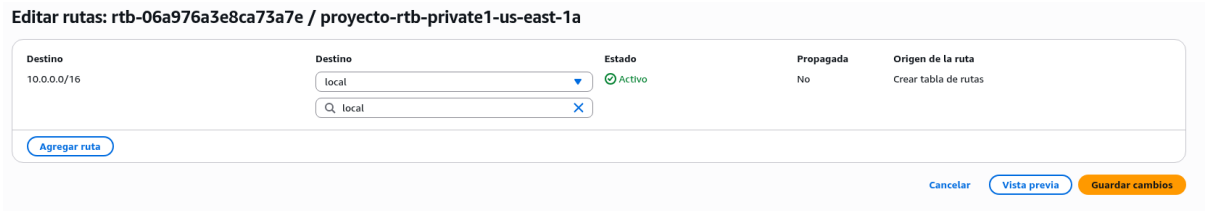


Figura 44. Eliminación de la ruta a internet de la tabla privada.



Figura 45. La tabla privada vuelve a tener solo la ruta local.

Repetimos el ping a www.google.com desde la privada y se pierden todos los paquetes. Ya no tiene salida a internet, pero sigue siendo accesible desde la pública por la red interna.

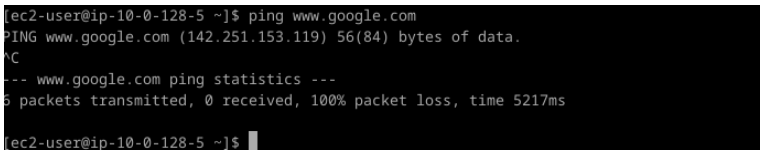


Figura 46. El servidor privado vuelve a estar sin salida a internet.

Nivel medio: WordPress con base de datos remota

El objetivo de este nivel es instalar Apache, PHP y WordPress en el servidor público y hacer que WordPress use la MariaDB del servidor privado en lugar de una base de datos local.

A diferencia de cloud-4, donde todo estaba en la misma máquina, aquí WordPress se conecta a la MariaDB del servidor privado a través de la red interna de la VPC. Así la base de datos nunca queda expuesta a internet y el único que puede hablar con ella es el servidor web.

1. Descripción general de la arquitectura

Partimos de la arquitectura del nivel básico. En la subred pública está la instancia con Apache, PHP y WordPress, que es la parte que ve el usuario, y en la subred privada la instancia con MariaDB, sin IP pública, a la que desde internet no se puede llegar directamente.

La instancia pública tiene dos interfaces de red: una en la subred pública (10.0.0.4), por la que entra el tráfico web y el SSH, y otra en la subred privada (10.0.128.4), por la que sale hacia la base de datos. Por eso más adelante el usuario de MariaDB se restringe a la 10.0.128.4.

Subred pública (10.0.0.0/20)	
EC2 Public	10.0.0.4 / 10.0.128.4 (segunda interfaz)
IP Pública	54.209.9.51
Servicios	Apache (httpd), PHP 8.5, WordPress Core
SG	grupo-publico

Subred privada (10.0.128.0/20)	
EC2 Private	10.0.128.5
IP Pública	Ninguna
Servicios	MariaDB Server 10.5
SG	grupo-privado

2. Acceso a la instancia pública

2.1. Entorno local

Trabajamos desde Ubuntu en WSL2. Con wsl --status y wsl -l -v comprobamos que la distribución está instalada y en ejecución con la versión 2, y abrimos su terminal para conectarnos desde ahí.

```
wsl --status
wsl -l -v
```

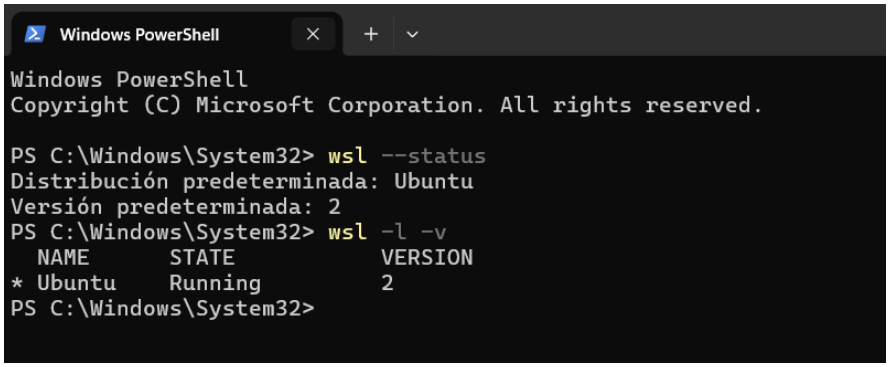


Figura 47. Estado de WSL2 desde PowerShell.



Figura 48. Terminal de Ubuntu en WSL2.

2.2. Conectar por SSH

Nos conectamos a la instancia pública con el par de claves de la asignatura y su IP pública. Como es la primera vez que nos conectamos desde este equipo, SSH pide confirmar la huella del servidor; al aceptarla aparece el banner de Amazon Linux 2023.

```
ssh -i ~/.ssh/keypair-cdist.pem ec2-user@54.209.9.51
```

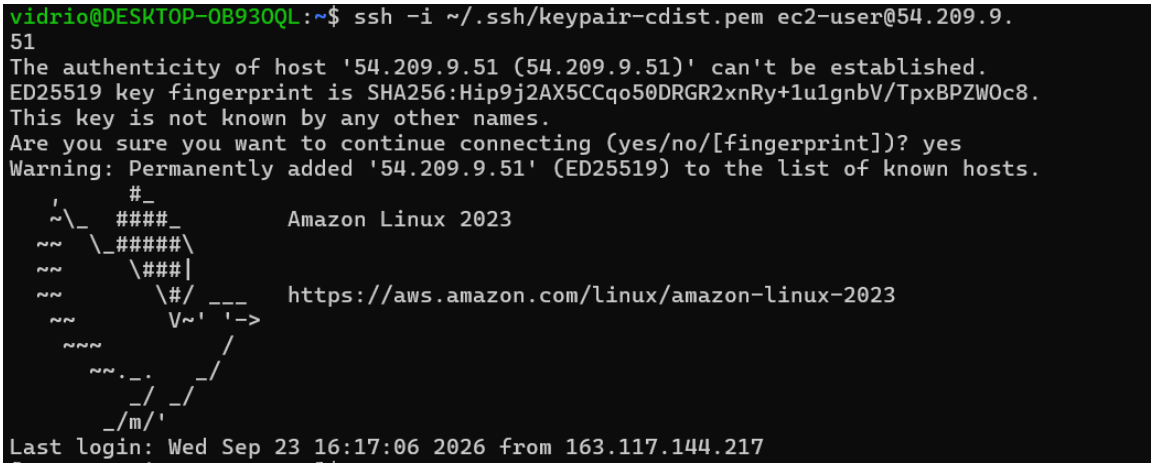


Figura 49. Conexión SSH al servidor público.

3. Servidor web Apache

3.1. Arrancar y habilitar Apache

Con Apache ya instalado, lo arrancamos y lo habilitamos para que se inicie solo cada vez que arranque la instancia. El enable crea el enlace simbólico en “multi-user.target”, y en el status aparece el servicio “active (running)” con varios procesos httpd, que son los workers que atienden las peticiones.

```
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
```

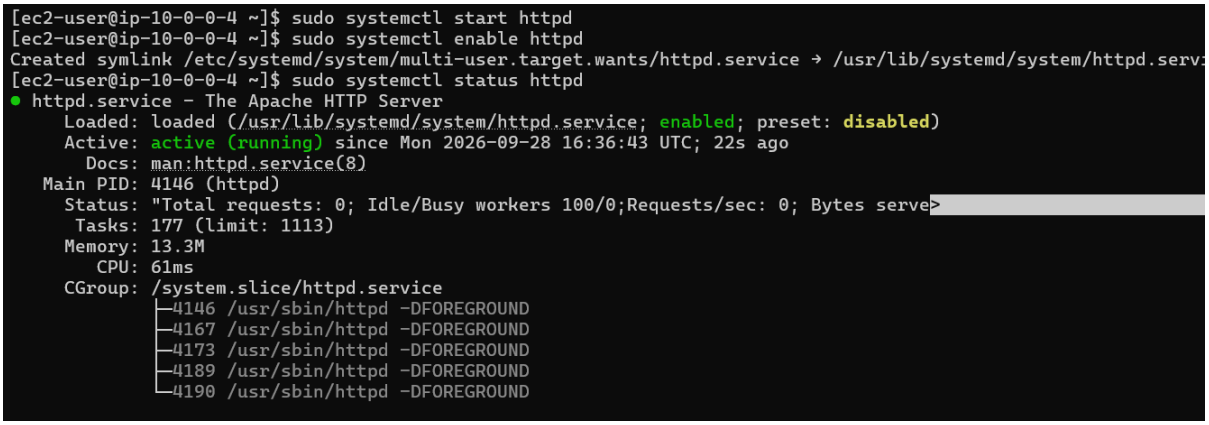


Figura 50. Apache arrancado y habilitado.

3.2. Grupo apache y configuración de Apache

Añadimos ec2-user al grupo apache para poder escribir en la carpeta web. Los cambios de grupo no se aplican hasta abrir una sesión nueva, así que salimos y volvemos a conectarnos. Aprovechamos para editar httpd.conf y permitir que Apache procese los ficheros .htaccess que usa WordPress (AllowOverride All en la sección de /var/www/html). Con groups comprobamos que ec2-user ya pertenece al grupo apache.

```
sudo usermod -a -G apache ec2-user
exit
ssh -i ~/.ssh/keypair-cdist.pem ec2-user@54.209.9.51
sudo nano /etc/httpd/conf/httpd.conf
groups
```

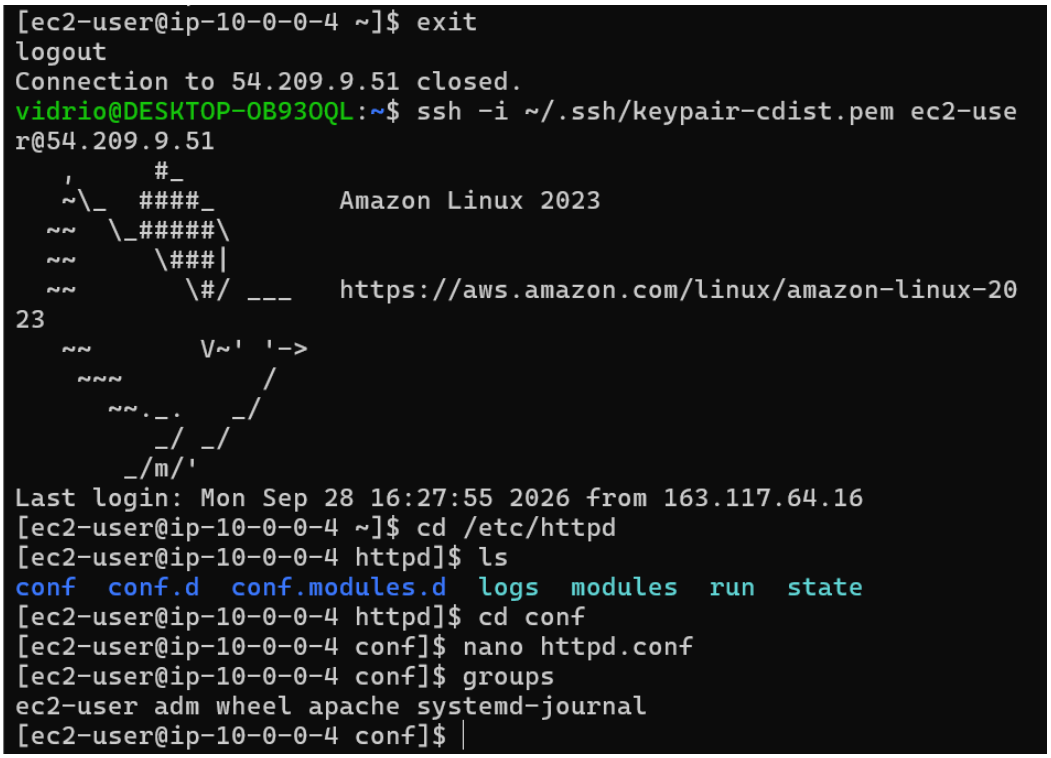


Figura 51. Reconexión, edición de httpd.conf y comprobación de grupos.

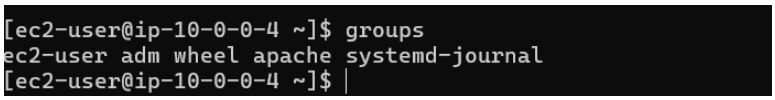


Figura 52. ec2-user ya pertenece al grupo apache.

4. Permisos y comprobación de PHP

4.1. Permisos de la carpeta web

Al listar /var/www/html se ve que la carpeta pertenece a root, por lo que todavía no podemos escribir en ella.

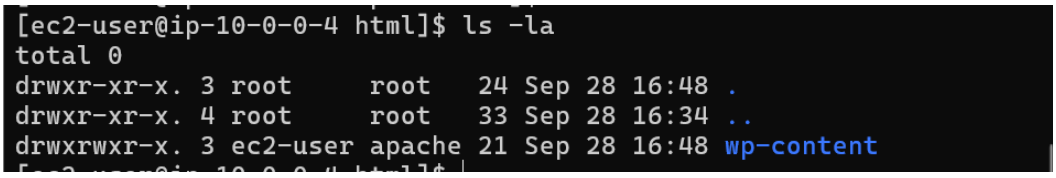


Figura 53. /var/www/html todavía pertenece a root.

Por eso los primeros intentos de crear ficheros dan “Permission denied” (y el cp falla porque en ese momento aún no habíamos descargado WordPress). Para solucionarlo ponemos ec2-user como propietario y apache como grupo. A las carpetas les damos 2775, donde el 2 (SGID) hace que todo lo nuevo herede el grupo apache, y a los ficheros 0664. Así tanto nosotros como Apache podemos escribir, que es lo que necesita WordPress para subir imágenes o instalar plugins. Después los comandos ya funcionan, incluido añadir

FS_METHOD en direct para que WordPress escriba directamente en disco en vez de pedir datos de FTP.

```
sudo chown -R ec2-user:apache /var/www/html/
sudo chmod 2775 /var/www/html && find /var/www/html -type d -exec sudo
chmod 2775 {} \;
find /var/www/html -type f -exec sudo chmod 0664 {} \;
echo "define('FS_METHOD', 'direct');" >> /var/www/html/wp-config.php
```

```
[ec2-user@ip-10-0-0-4 ~]$ echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php
-bash: /var/www/html/phpinfo.php: Permission denied
[ec2-user@ip-10-0-0-4 ~]$ echo "define('FS_METHOD', 'direct');" >> /var/www/html/wp-config.php
-bash: /var/www/html/wp-config.php: Permission denied
[ec2-user@ip-10-0-0-4 ~]$ cp -r wordpress/* /var/www/html/
cp: cannot stat 'wordpress/*': No such file or directory
[ec2-user@ip-10-0-0-4 ~]$ sudo chown -R ec2-user:apache /var/www/html/
sudo chmod 2775 /var/www/html && find /var/www/html -type d -exec sudo chmod 2775 {} \;
find /var/www/html -type f -exec sudo chmod 0664 {} \;
[ec2-user@ip-10-0-0-4 ~]$ echo "define('FS_METHOD', 'direct');" >> /var/www/html/wp-config.php
[ec2-user@ip-10-0-0-4 ~]$ echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php
[ec2-user@ip-10-0-0-4 ~]$
```

Figura 54. Primeros intentos con Permission denied y ajuste de permisos.

```
[ec2-user@ip-10-0-0-4 ~]$ sudo chown -R ec2-user:apache /var/www/html/
sudo chmod 2775 /var/www/html && find /var/www/html -type d -exec sudo chmod 2775 {} \;
find /var/www/html -type f -exec sudo chmod 0664 {} \;
[ec2-user@ip-10-0-0-4 ~]$ echo "define('FS_METHOD', 'direct');" >> /var/www/html/wp-config.php
[ec2-user@ip-10-0-0-4 ~]$ echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php
[ec2-user@ip-10-0-0-4 ~]$
```

Figura 55. Los mismos comandos, ya sin errores.

4.2. Probar PHP

Para comprobar que Apache ejecuta PHP creamos un fichero con phpinfo() y lo abrimos en el navegador. Aparece PHP 8.5.10 funcionando con FPM/FastCGI, y entre las extensiones cargadas están mysqli y pdo_mysql, que son las que usa WordPress para conectarse a MariaDB.

```
echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php
http://54.209.9.51/phpinfo.php
```

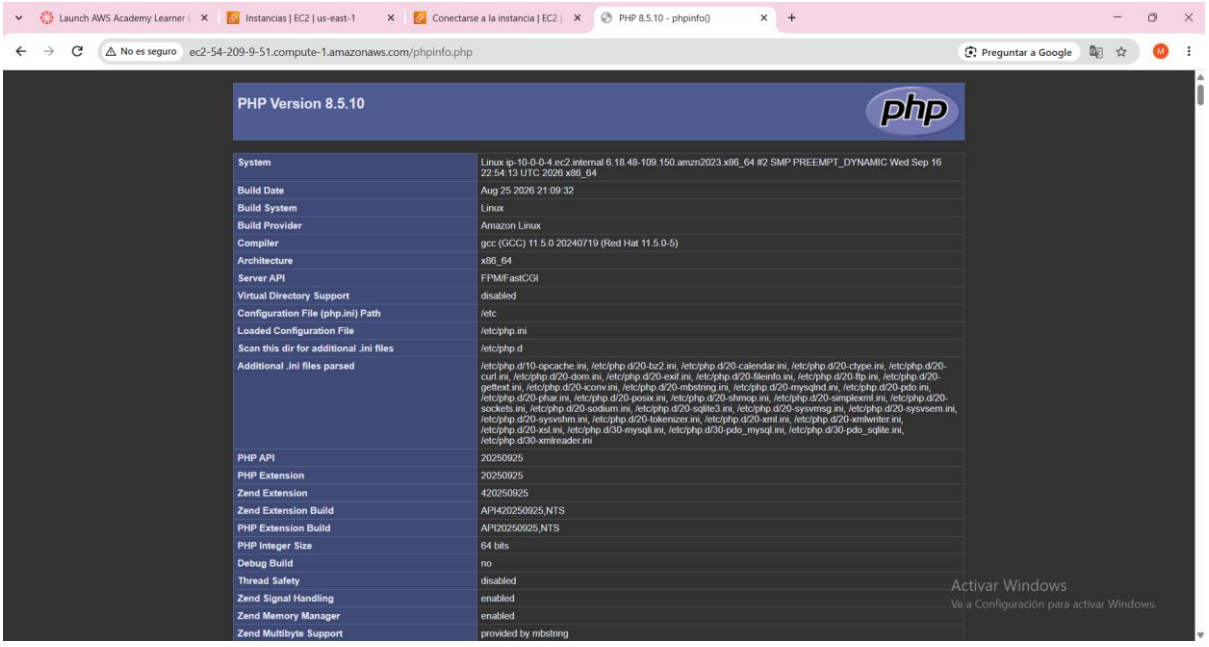


Figura 56. phpinfo() en el navegador.

Build Date	Aug 25 2026 21:09:32
Build System	Linux
Build Provider	Amazon Linux
Compiler	gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-5)
Architecture	x86_64
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	/etc/php.d
Additional .ini files parsed	/etc/php.d/10-opcache.ini, /etc/php.d/20-bz2.ini, /etc/php.d/20-calendar.ini, /etc/php.d/20-ctype.ini, /etc/php.d/20-curl.ini, /etc/php.d/20-dom.ini, /etc/php.d/20-exif.ini, /etc/php.d/20-fileinfo.ini, /etc/php.d/20-ftp.ini, /etc/php.d/20-gettext.ini, /etc/php.d/20-iconv.ini, /etc/php.d/20-mbstring.ini, /etc/php.d/20-mysqlnd.ini, /etc/php.d/20-pdo.ini, /etc/php.d/20-phar.ini, /etc/php.d/20-posix.ini, /etc/php.d/20-shmop.ini, /etc/php.d/20-simplexml.ini, /etc/php.d/20-sockets.ini, /etc/php.d/20-sodium.ini, /etc/php.d/20-sqlite3.ini, /etc/php.d/20-sysvmsg.ini, /etc/php.d/20-sysvsem.ini, /etc/php.d/20-sysvshm.ini, /etc/php.d/20-tokenizer.ini, /etc/php.d/20-xml.ini, /etc/php.d/20-xmlwriter.ini, /etc/php.d/20-xsl.ini, /etc/php.d/30-mysqli.ini, /etc/php.d/30-pdo_mysql.ini, /etc/php.d/30-pdo_sqlite.ini, /etc/php.d/30-xmlreader.ini
PHP API	20250925
PHP Extension	20250925
Zend Extension	420250925
Zend Extension Build	API420250925,NTS
PHP Extension Build	API20250925,NTS
PHP Integer Size	64 bits
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	available, disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, compress.bzip2, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	zlib.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk, bzip2.*, convert.iconv.*

Figura 57. Detalle de phpinfo(): PHP 8.5.10 con FPM/FastCGI.

Una vez comprobado lo borramos, porque deja a la vista mucha información del servidor.

```
rm /var/www/html/phpinfo.php
```

```
[ec2-user@ip-10-0-0-4 ~]$ rm /var/www/html/phpinfo.php
```

Figura 58. Borrado de phpinfo.php.

5. Descargar WordPress

Descargamos la última versión de WordPress desde la web oficial; la descarga, de unos 34 MB, se completa sin problemas. Después la descomprimimos y con ls comprobamos que se ha creado la carpeta wordpress junto al .tar.gz.

```
cd ~
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
ls
```

```
[ec2-user@ip-10-0-0-4 ~]$ cd ~
wget https://wordpress.org/latest.tar.gz
--2026-09-28 21:56:12--  https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 66.6.42.252, 2620:109:b00a::4206:2afc
Connecting to wordpress.org (wordpress.org)|66.6.42.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 35362548 (34M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz                       100%[=============================================================================>]  33.72M  95.5MB/s    in 0.4s

2026-09-28 21:56:12 (95.5 MB/s) - 'latest.tar.gz' saved [35362548/35362548]

[ec2-user@ip-10-0-0-4 ~]$
```

Figura 59. Descarga de WordPress con wget.



Figura 60. WordPress descomprimido en el home.

6. Base de datos en el servidor privado

A diferencia de cloud-4, en la pública no instalamos el servidor de MariaDB, porque la base de datos está en el privado. WordPress necesita una base de datos propia y un usuario dedicado, nunca se conecta como root.

6.1. Conectar al servidor privado

Igual que en el nivel básico, entramos al servidor privado usando la pública como bastion host: desde ella hacemos SSH a la 10.0.128.5. El prompt cambia a ip-10-0-128-5, y en el “Last login” se ve que la conexión anterior también llegó desde 10.0.128.4, la interfaz privada de la pública, que es justo por donde tiene que entrar.

```
ssh -i keypair-cdist.pem ec2-user@10.0.128.5
```

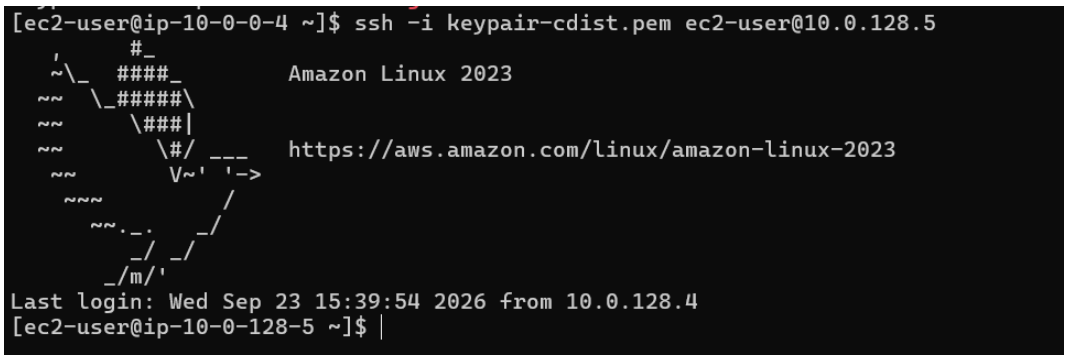


Figura 61. Conexión al servidor privado desde el público.

6.2. Arrancar y asegurar MariaDB

MariaDB ya estaba instalado del nivel básico. Como el laboratorio se había parado entre sesiones, volvemos a arrancar el servicio, comprobamos que queda habilitado para iniciarse solo y repasamos la configuración con mysql_secure_installation.

```
sudo systemctl start mariadb
sudo systemctl enable mariadb
sudo mysql_secure_installation
```

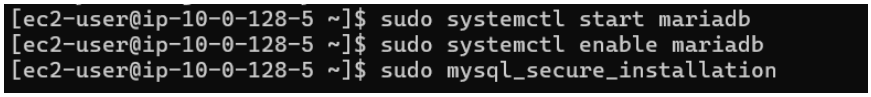


Figura 62. Arranque de MariaDB en el servidor privado.

6.3. Crear el usuario y dar privilegios

Entramos como root y creamos el usuario publico. En vez de localhost ponemos 10.0.128.4, que es la IP con la que llegan las conexiones desde el servidor web, así que el usuario solo puede entrar desde ahí. Le damos todos los permisos únicamente sobre wordpress-db, recargamos los privilegios y con SHOW DATABASES comprobamos que la base de datos existe.

```
CREATE USER 'publico'@'10.0.128.4' IDENTIFIED BY 'TU_PASSWORD_PUBLICO';
GRANT ALL PRIVILEGES ON `wordpress-db`.* TO 'publico'@'10.0.128.4';
FLUSH PRIVILEGES;
SHOW DATABASES;
```

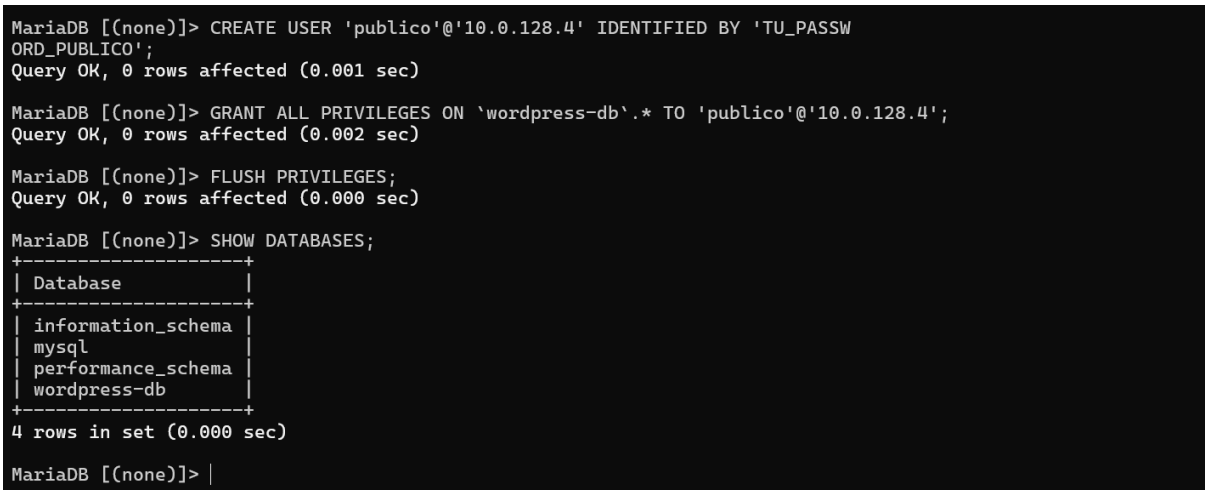


Figura 63. Usuario publico restringido a 10.0.128.4 y base de datos wordpress-db.

7. Configuración de WordPress (wp-config.php)

7.1. Datos de la base de datos

De vuelta en la pública, copiamos wp-config-sample.php a wp-config.php y lo editamos con nano. Primero rellenamos el nombre de la base de datos, el usuario y la contraseña; en esta captura DB_HOST todavía tiene el valor por defecto, localhost.

```
cp wordpress/wp-config-sample.php wordpress/wp-config.php
nano wordpress/wp-config.php
```

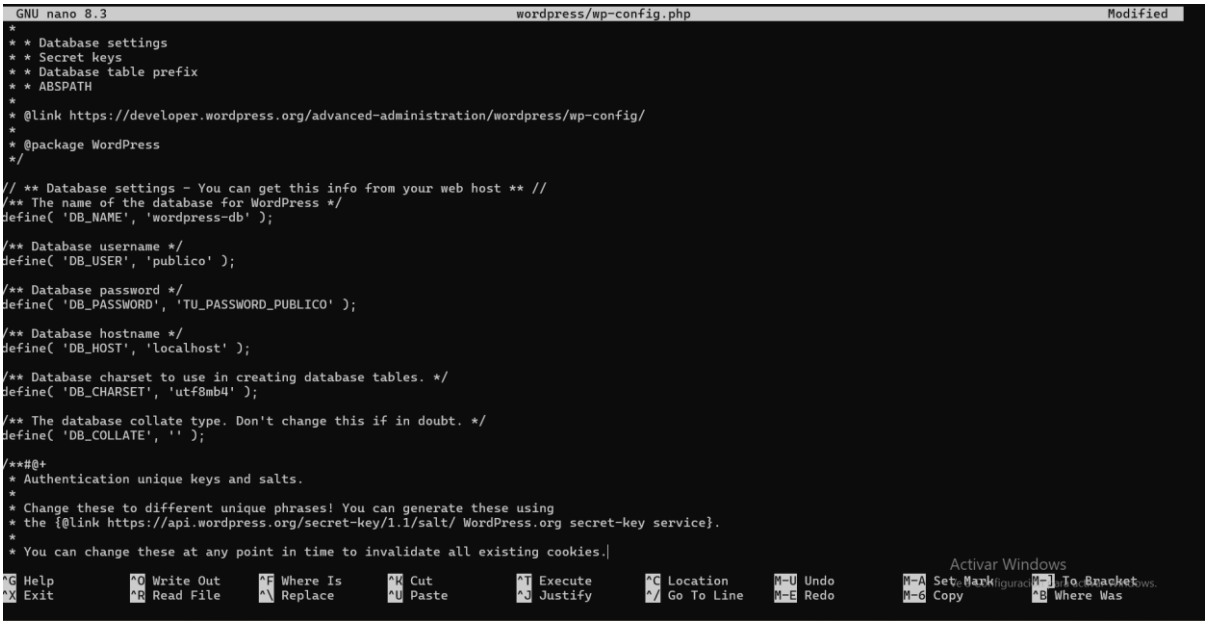


Figura 64. wp-config.php con los datos de la base de datos (DB_HOST aún en localhost).

7.2. Claves de seguridad

Sustituimos el bloque de claves (AUTH_KEY, SECURE_AUTH_KEY…) por las que genera la web oficial de WordPress. Se usan para firmar las cookies de sesión, así que cada instalación debe tener las suyas.

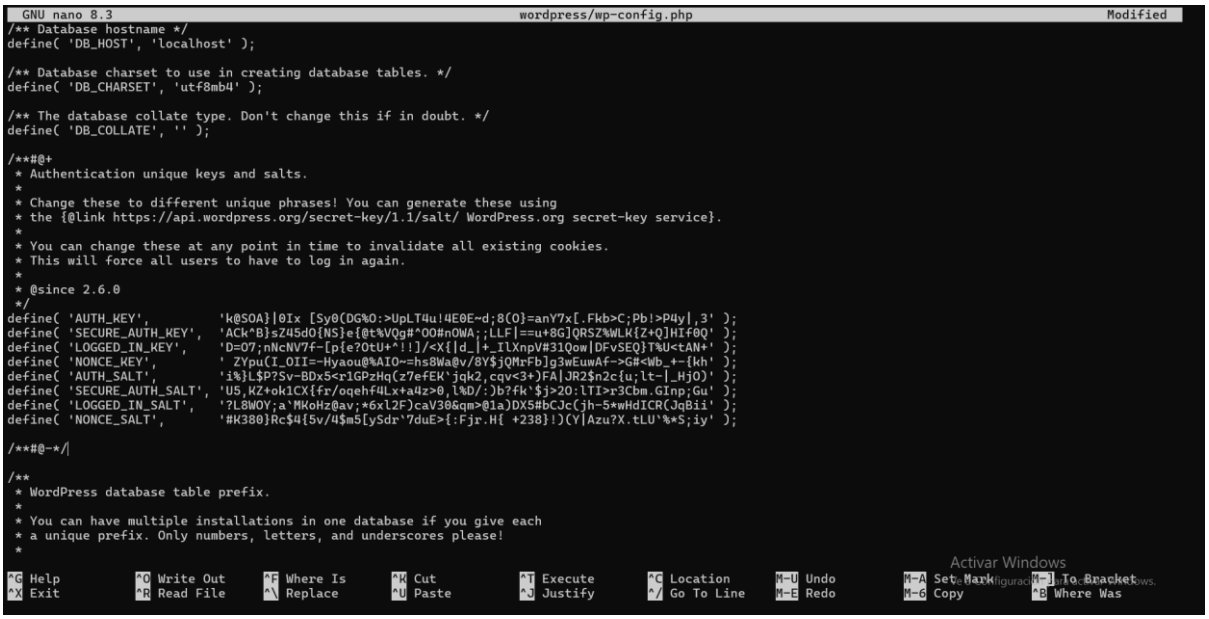


Figura 65. Claves y salts de autenticación.

7.3. Apuntar a la base de datos remota

El cambio importante respecto a cloud-4 está en DB_HOST: en vez de localhost ponemos 10.0.128.5, la IP del servidor privado, para que WordPress use la base de datos remota a través de la red interna de la VPC. Por último copiamos el contenido de la carpeta wordpress a /var/www/html, que es la que sirve Apache.

```
define( 'DB_HOST', '10.0.128.5' );
cp -r wordpress/* /var/www/html/
```

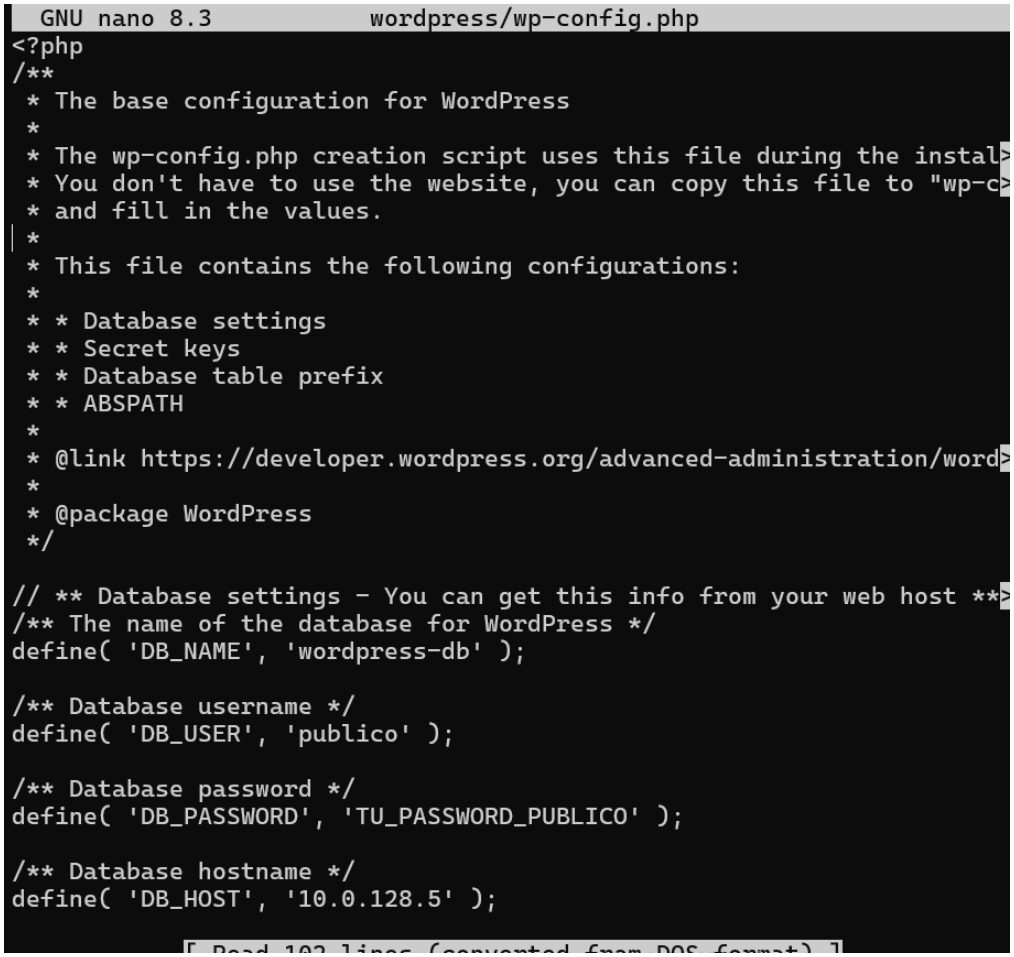


Figura 66. wp-config.php apuntando a 10.0.128.5.

8. Comprobación del funcionamiento

8.1. Blog funcionando

Abrimos en el navegador la dirección de la pública y completamos el asistente de instalación con el título del blog, el usuario y la contraseña. Al instalarse, WordPress crea sus tablas en wordpress-db, en el servidor privado. En la captura se ve el blog con la entrada de ejemplo “Hello world!” y la barra de administración arriba, porque ya hemos iniciado sesión.

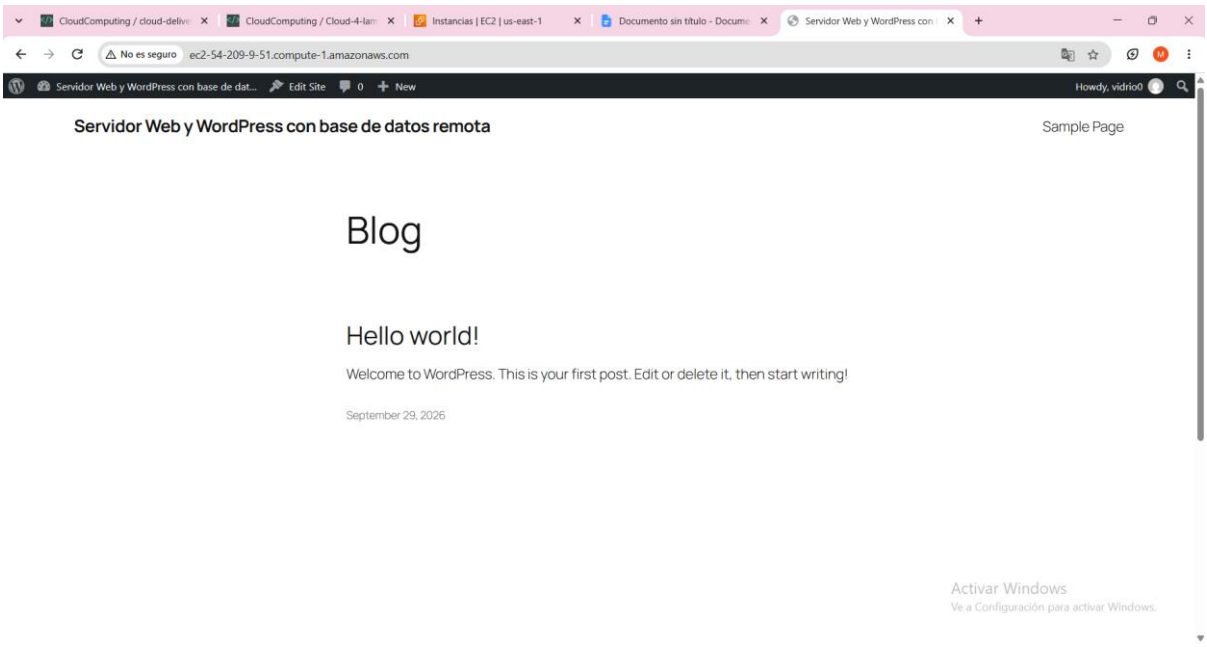


Figura 67. Blog funcionando con la base de datos remota.

8.2. Permisos del usuario

En el privado consultamos los permisos de publico. Salen dos líneas: USAGE, que solo le permite conectarse, y ALL PRIVILEGES limitado a wordpress-db, así que no tiene acceso a ninguna otra base de datos.

```
SHOW GRANTS FOR 'publico'@'10.0.128.4';
```

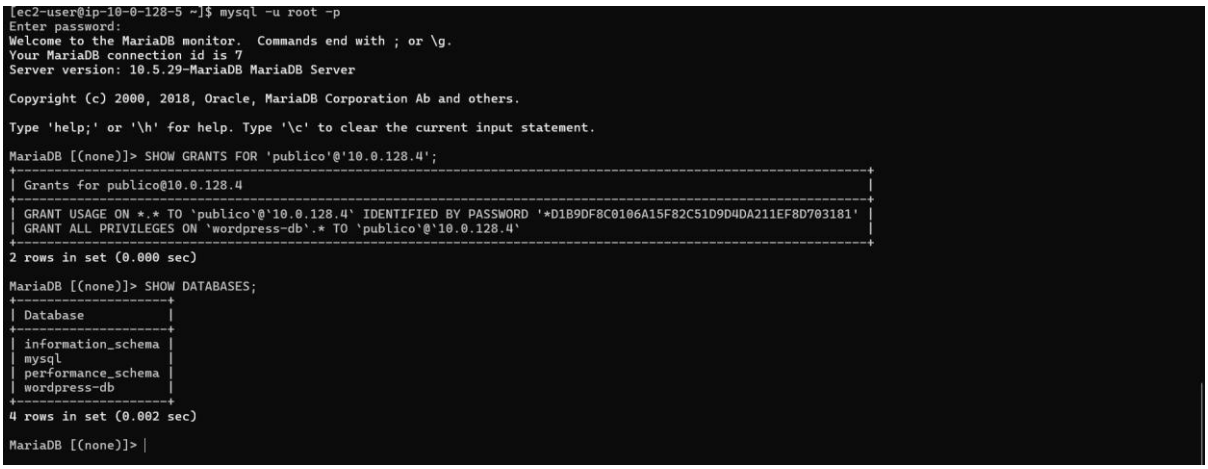


Figura 68. Permisos del usuario publico.

8.3. Separación de servicios

Para dejar claro que la base de datos no está en el servidor web, consultamos el estado de MariaDB en las dos máquinas. En la pública devuelve “Unit mariadb.service could not be found”, porque ni siquiera está instalado.

```
sudo systemctl status mariadb
```

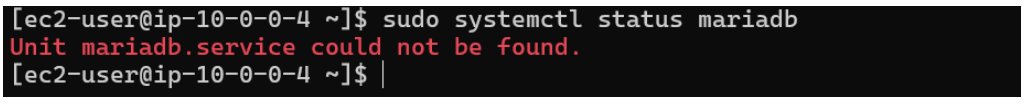


Figura 69. El servidor público no tiene MariaDB instalado.

En la privada, en cambio, aparece “active (running)” con el mensaje “Taking your SQL requests now…”. Todo el contenido del blog está guardado en el servidor privado, que sigue sin ser accesible desde internet.



Figura 70. MariaDB en ejecución en el servidor privado.

Nivel avanzado (opción A): WordPress sobre Amazon RDS

El objetivo de este nivel es sustituir el MariaDB instalado a mano en el servidor privado por una base de datos gestionada de Amazon RDS y hacer que WordPress se conecte a ella.

Amazon RDS es el servicio de bases de datos gestionadas de AWS. En lugar de una EC2 en la que nosotros instalamos, arrancamos y mantenemos MariaDB, RDS nos da la base de datos ya funcionando y se encarga del sistema operativo, los parches y las copias de seguridad. Desde fuera solo vemos un “endpoint”, un nombre DNS al que conectarse.

Reutilizamos todo lo de los niveles anteriores: la VPC proyecto-vpc, sus subredes y servidor-web-publico con Apache, PHP y WordPress. Lo único que cambia es dónde vive la base de datos y, en wp-config.php, la línea DB_HOST.

1. Security Group para RDS

Creamos grupo-rds en proyecto-vpc, que es el que llevará la base de datos. Tiene dos reglas de entrada, las dos MySQL/Aurora en el puerto 3306: una con origen grupo-publico y otra con origen grupo-privado. Las reglas de salida se quedan como vienen.

Ponemos las dos porque el servidor web tiene dos interfaces y cada una lleva un grupo distinto: enX0 (10.0.0.4) va con grupo-publico y enX1 (10.0.128.4) con grupo-privado. Por qué interfaz sale el tráfico hacia RDS depende de la subred en la que acabe la base de datos. Si cae en 10.0.128.0/20 se usa la ruta directa de enX1 que vimos en el nivel básico; en cualquier otra subred sale por la ruta por defecto, enX0. Con las dos reglas funciona en ambos casos, y en ninguno se abre RDS a internet.

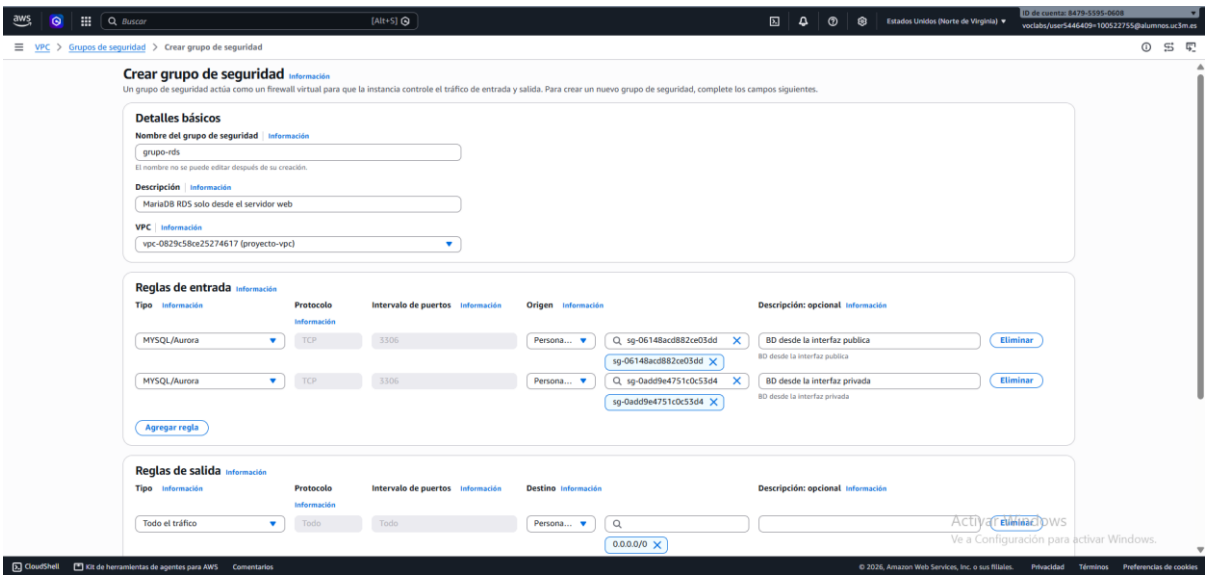


Figura 71. Creación de grupo-rds con las dos reglas MySQL/Aurora.

2. Grupo de subredes de base de datos

RDS necesita un grupo de subredes que le diga dónde puede colocar la base de datos, y AWS obliga a que tenga subredes en al menos dos zonas de disponibilidad. Creamos grupo-subredes-rds en proyecto-vpc con las dos subredes privadas: proyecto-subnet-private1-us-east-1a (10.0.128.0/20) y proyecto-subnet-private2-us-east-1b (10.0.144.0/20). Es la primera

vez que usamos la subred privada de us-east-1b, que se creó en el nivel básico precisamente para esto.

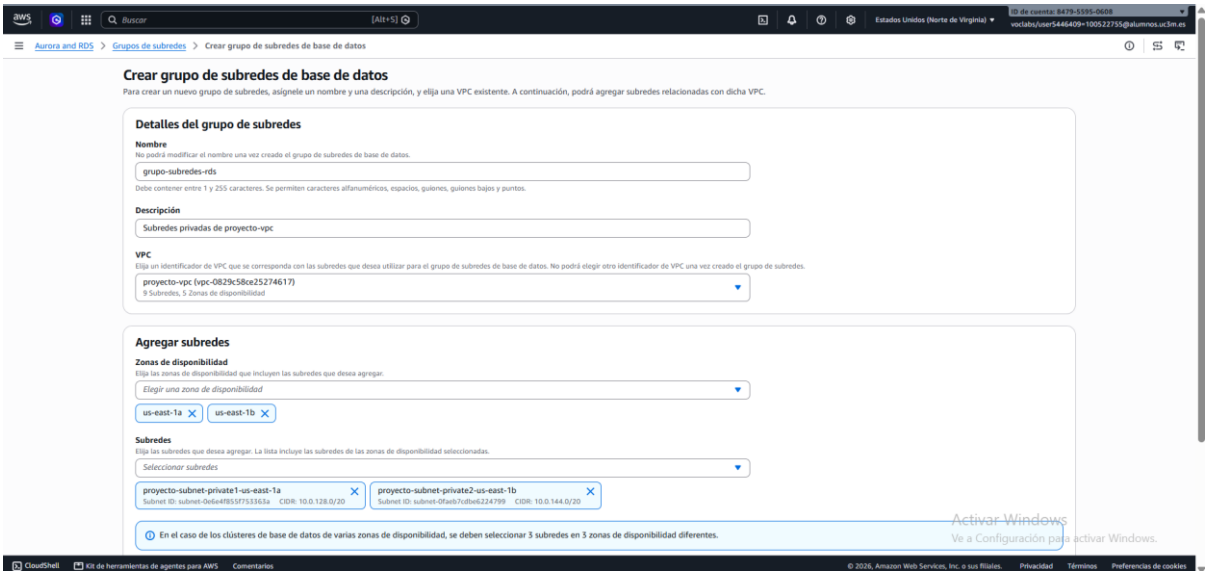


Figura 72. Grupo de subredes con las dos subredes privadas.

Antes de crearlo, la tabla de “Subredes seleccionadas” confirma las dos subredes, cada una en su zona. El aviso azul sobre tres subredes solo se aplica a clústeres Multi-AZ, que no usamos (el Learner Lab tampoco los permite).

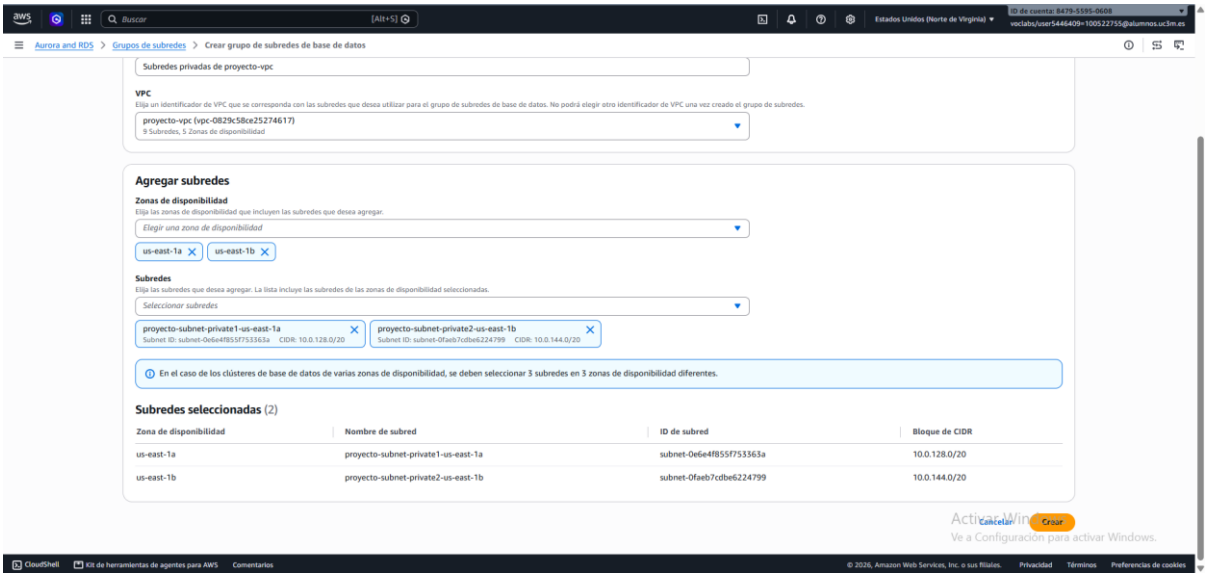


Figura 73. Resumen de las subredes seleccionadas antes de crear el grupo.

3. Crear la instancia RDS

3.1. Motor y plantilla

En RDS → “Crear base de datos” elegimos “Configuración completa”, para poder fijar nosotros la red, y el motor MariaDB, que es compatible con MySQL y con lo que ya usaba WordPress. Como plantilla, “Capa gratuita”: crea una sola instancia sin réplica, que es lo único que permite el Learner Lab.

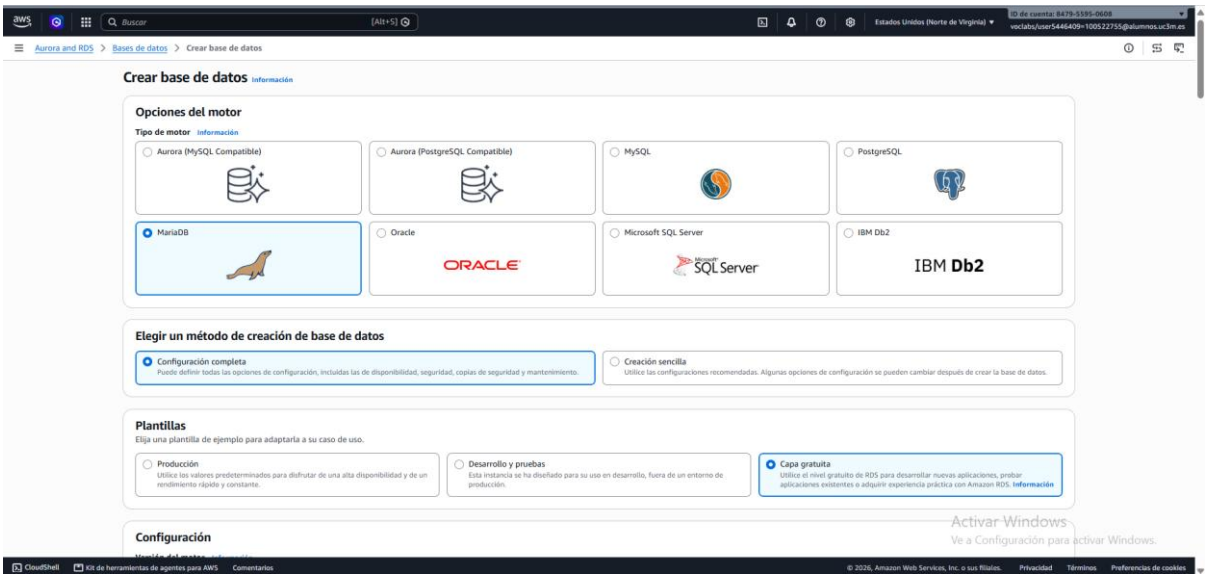


Figura 74. Motor MariaDB, configuración completa y plantilla de capa gratuita.

3.2. Identificador y credenciales

Dejamos la versión que propone la consola, MariaDB 11.8.8. Como identificador ponemos wordpress-rds (forma parte del endpoint) y como usuario maestro admin. La contraseña la elegimos nosotros (“Autoadministrado”) en lugar de guardarla en Secrets Manager, para no complicar el acceso.

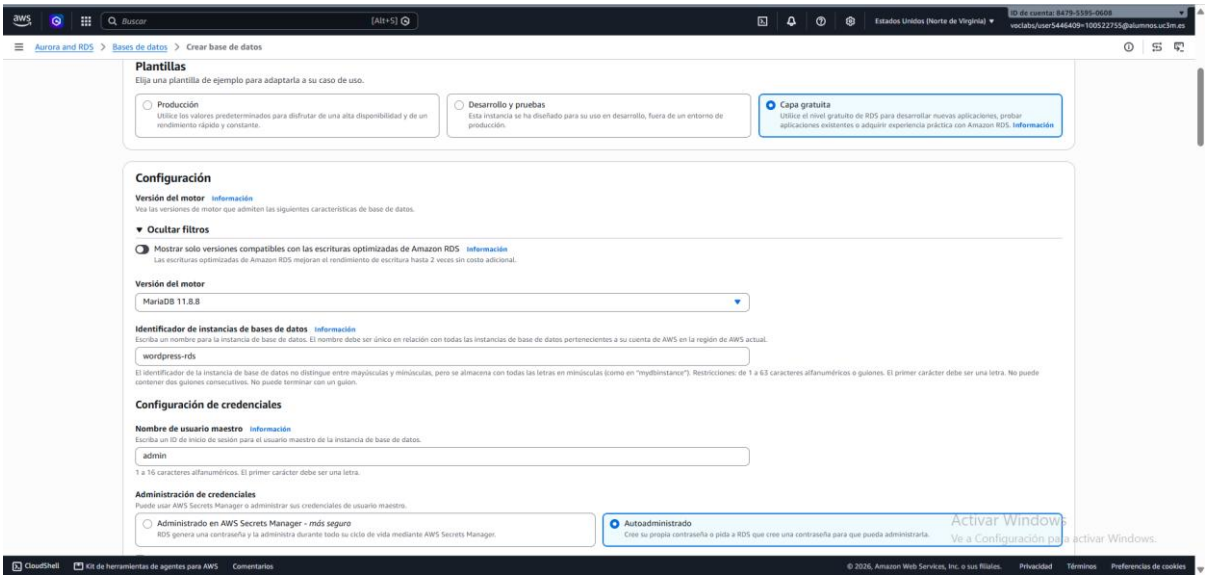


Figura 75. Versión MariaDB 11.8.8, identificador wordpress-rds y usuario maestro.

3.3. Clase de instancia y almacenamiento

Autenticación solo con contraseña, clase db.t4g.micro (2 vCPU y 1 GiB de RAM, suficiente para un blog) y 20 GiB de SSD de uso general gp2. El Learner Lab solo admite clases pequeñas y almacenamiento gp2; además desmarcamos la monitorización mejorada, que tampoco permite.

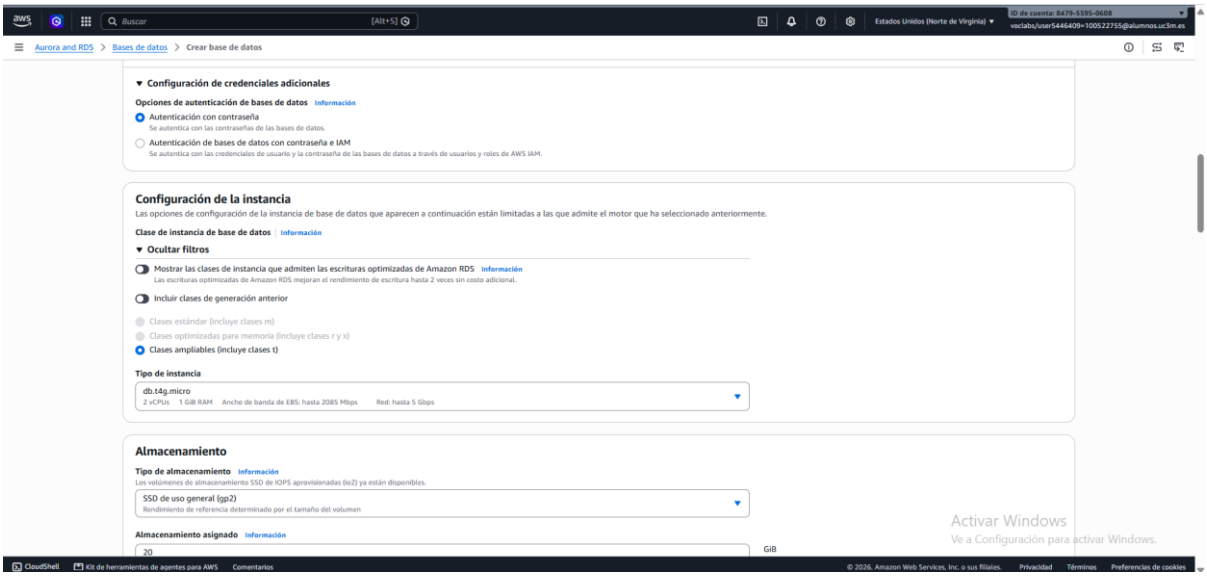


Figura 76. Clase db.t4g.micro y almacenamiento gp2.

3.4. Conectividad

Esta es la parte más importante de la configuración. No conectamos ningún recurso EC2 desde el asistente, porque los grupos de seguridad ya los hemos preparado nosotros. Elegimos proyecto-vpc y grupo-subredes-rds (2 subredes, 2 zonas), con “Acceso público” en “No”: RDS no recibe IP pública y solo se puede llegar a ella desde dentro de la VPC. Como grupo de seguridad elegimos el existente grupo-rds. En la configuración adicional damos wordpressdb como nombre de la base de datos inicial, para que RDS la cree vacía al arrancar.

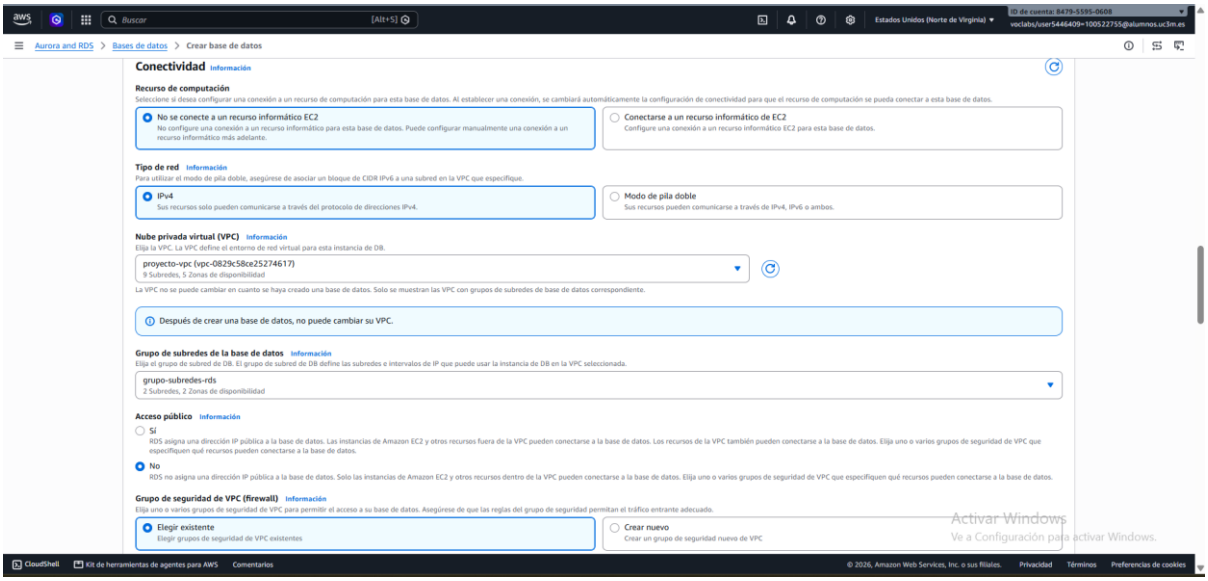


Figura 77. Conectividad: proyecto-vpc, grupo-subredes-rds y acceso público desactivado.

3.5. Instancia disponible

A los pocos minutos wordpress-rds aparece como “Disponible”. Como en la zona de disponibilidad dejamos “Sin preferencia”, AWS la ha colocado en us-east-1b, es decir, en la subred privada 10.0.144.0/20. En la pestaña “Conectividad y seguridad” están el endpoint, el puerto 3306, la VPC y el grupo de subredes, grupo-rds activo y “Accesible públicamente: No”. Arriba se ve también wordpressdb como nombre de la base de datos y la puerta de enlace a internet desactivada.

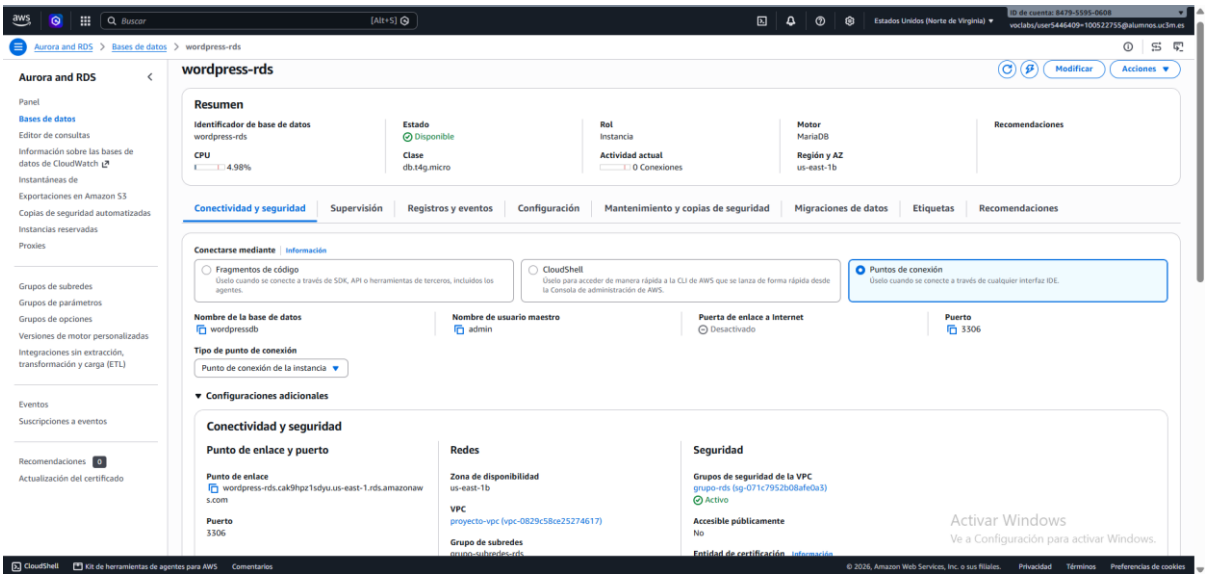


Figura 78. wordpress-rds disponible en us-east-1b, sin acceso público.

4. Conexión a RDS desde el servidor web

4.1. Endpoint en una variable

Entramos por SSH en servidor-web-publico y guardamos el endpoint en la variable RDS. Al añadirla a ~/.bashrc se carga sola en cada sesión, y así no tenemos que escribir el nombre completo en cada comando. El aviso de “A newer release” solo indica que hay una versión más reciente de Amazon Linux.

```
echo 'export RDS=wordpress-rds.cak9hpz1sdyu.us-east-1.rds.amazonaws.com'
>> ~/.bashrc
source ~/.bashrc
echo $RDS
```

```
vidrio@DESKTOP-OB93OQL:~/.ssh$ ssh -i keypair-cdist.pem ec2-user@54.209.9.51

A newer release of "Amazon Linux" is available.
  Version 2023.12.20260928:
  Version 2023.12.20260930:
Run "/usr/bin/dnf check-release-update" for full release and version update info
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Tue Sep 29 21:25:45 2026 from 188.26.197.45
[ec2-user@ip-10-0-0-4 ~]$ echo 'export RDS=wordpress-rds.cak9hpz1sdyu.us-east-1.rds.amazonaws.com' >> ~/.bashrc
[ec2-user@ip-10-0-0-4 ~]$ source ~/.bashrc
[ec2-user@ip-10-0-0-4 ~]$ echo $RDS
wordpress-rds.cak9hpz1sdyu.us-east-1.rds.amazonaws.com
[ec2-user@ip-10-0-0-4 ~]$
```

Figura 79. Conexión SSH al servidor web y endpoint de RDS en una variable.

4.2. Resolución del endpoint

Con getent hosts vemos a qué IP apunta el endpoint: 10.0.148.38, una dirección privada dentro de 10.0.144.0/20, que coincide con la zona us-east-1b de la consola. Desde fuera de la VPC esa IP no lleva a ningún sitio. Como el servidor web no tiene ruta específica para esa subred, el tráfico sale por enX0 y llega a RDS con origen grupo-publico, que es la primera regla de grupo-rds.

```
getent hosts $RDS
```

```
[ec2-user@ip-10-0-0-4 ~]$ getent hosts $RDS
10.0.148.38     wordpress-rds.cak9hpz1sdyu.us-east-1.rds.amazonaws.com
[ec2-user@ip-10-0-0-4 ~]$
```

Figura 80. El endpoint de RDS resuelve a una IP privada.

4.3. Entrar como usuario maestro

Nos conectamos con el cliente de MariaDB que ya teníamos instalado del nivel básico. Hace falta añadir --ssl: a partir de MariaDB 11.8, RDS obliga a que todas las conexiones vayan cifradas y rechaza las que no lo están. El monitor muestra la versión 11.8.8-MariaDB managed by https://aws.amazon.com/rds/.

```
mysql -h $RDS -u admin -p --ssl
```

```
[ec2-user@ip-10-0-0-4 ~]$ mysql -h $RDS -u admin -p --ssl
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 496
Server version: 11.8.8-MariaDB managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Figura 81. Conexión a RDS como usuario maestro con --ssl.

SELECT VERSION() confirma la versión y SHOW DATABASES muestra wordpressdb, que RDS creó al arrancar, junto a las bases de datos internas de MariaDB (mysql, sys, performance_schema…).

```
SELECT VERSION();
SHOW DATABASES;
```

```
MariaDB [(none)]> SELECT VERSION();
+----------------+
| VERSION()      |
+----------------+
| 11.8.8-MariaDB |
+----------------+
1 row in set (0.007 sec)

MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| innodb             |
| mysql              |
| performance_schema |
| sys                |
| wordpressdb        |
+--------------------+
6 rows in set (0.005 sec)

MariaDB [(none)]>
```

Figura 82. Versión del servidor y bases de datos de RDS.

4.4. Usuario de WordPress

WordPress no debe conectarse con admin, que tiene permisos sobre todo el servidor, así que creamos wordpress-user con permisos solo sobre wordpressdb. Como host ponemos 10.0.%: el usuario solo se acepta si la conexión viene de una IP de la VPC. No lo limitamos a una sola IP porque la de origen depende de la interfaz por la que salga el tráfico; el filtrado fino ya lo hace grupo-rds.

```
CREATE USER 'wordpress-user'@'10.0.%' IDENTIFIED BY
'TU_PASSWORD_WORDPRESS';
GRANT ALL PRIVILEGES ON `wordpressdb`.* TO 'wordpress-user'@'10.0.%';
FLUSH PRIVILEGES;
```

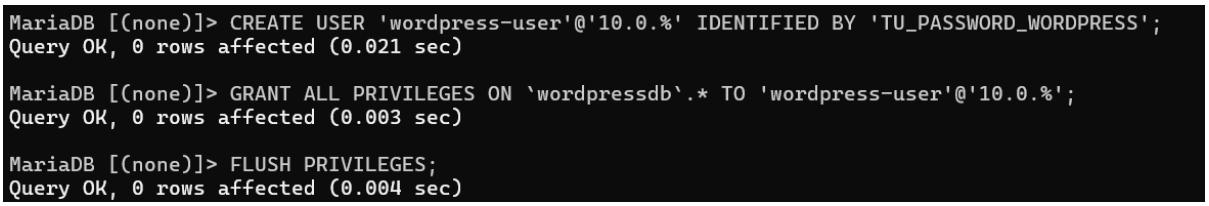


Figura 83. Creación de wordpress-user.

SHOW GRANTS devuelve las dos líneas que esperábamos: USAGE, que solo permite conectarse, y ALL PRIVILEGES limitado a wordpressdb.

```
SHOW GRANTS FOR 'wordpress-user'@'10.0.%';
```



Figura 84. Permisos de wordpress-user.

4.5. Probar el usuario nuevo

Repetimos la conexión con wordpress-user. Ahora solo aparecen information_schema y wordpressdb: el resto del servidor no lo ve.

```
mysql -h $RDS -u wordpress-user -p --ssl -e "SHOW DATABASES";
```

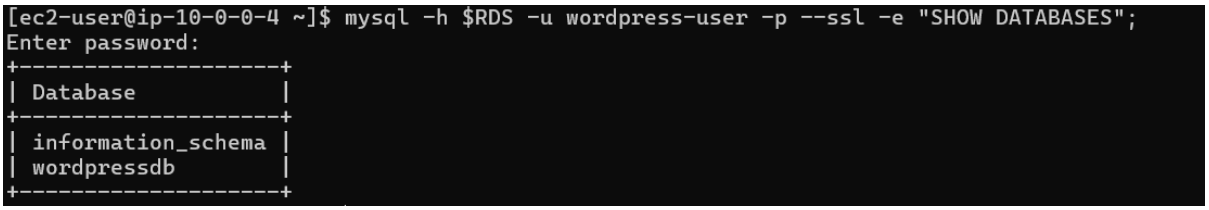


Figura 85. wordpress-user solo ve wordpressdb.

5. Apuntar WordPress a RDS

5.1. Copia de seguridad de la configuración

Antes de tocar nada guardamos una copia de wp-config.php. Si algo sale mal, basta con restaurarla para volver al nivel medio. Editamos el fichero que sirve Apache, el de /var/www/html.

```
cp /var/www/html/wp-config.php /var/www/html/wp-config.php.bak
nano /var/www/html/wp-config.php
```



Figura 86. Copia de seguridad de wp-config.php.

5.2. Nuevos datos de conexión

Cambiamos las cuatro líneas de la base de datos: wordpressdb, wordpress-user, su contraseña y, en DB_HOST, el endpoint de RDS en lugar de 10.0.128.5. Ponemos el nombre y no la IP porque, si AWS mueve la base de datos o la recupera tras un fallo, la IP puede cambiar pero el endpoint no. El resto del fichero, con las claves y “salts”, se queda igual.

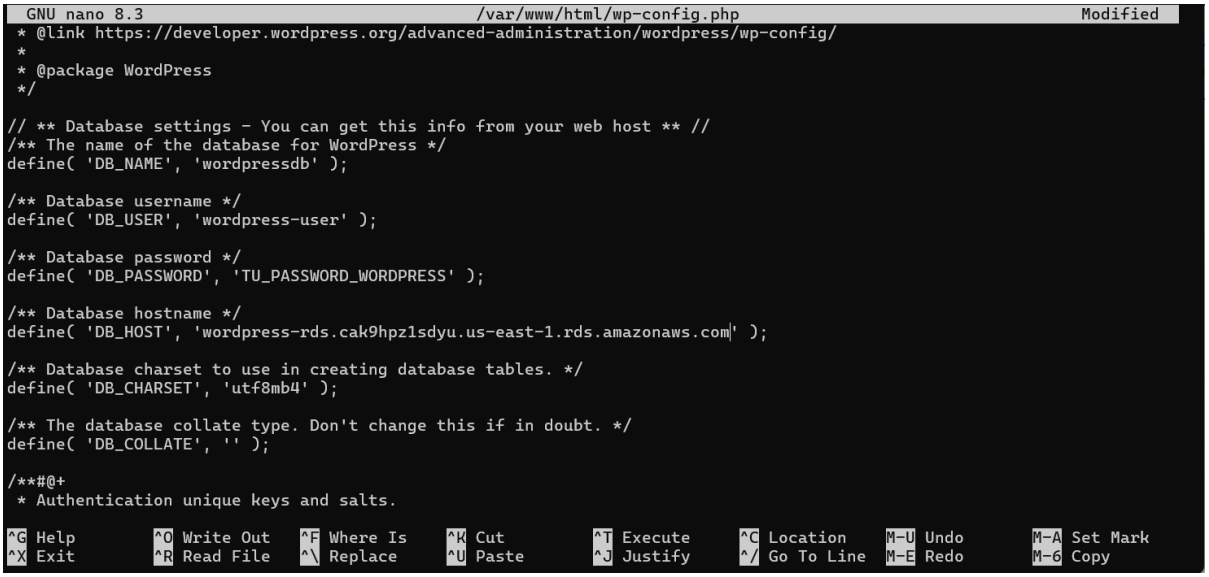


Figura 87. wp-config.php con DB_HOST apuntando a RDS.

5.3. Comprobación

Con grep comprobamos que las líneas DB_ han quedado bien y reiniciamos Apache.

```
grep "DB_" /var/www/html/wp-config.php
sudo systemctl restart httpd
```

```
[ec2-user@ip-10-0-0-4 ~]$ grep "DB_" /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpressdb' );
define( 'DB_USER', 'wordpress-user' );
define( 'DB_PASSWORD', 'TU_PASSWORD_WORDPRESS' );
define( 'DB_HOST', 'wordpress-rds.cak9hpz1sdyu.us-east-1.rds.amazonaws.com' );
define( 'DB_CHARSET', 'utf8mb4' );
define( 'DB_COLLATE', '' );
[ec2-user@ip-10-0-0-4 ~]$ sudo systemctl restart httpd
```

Figura 88. Comprobación de wp-config.php y reinicio de Apache.

6. Error de conexión y solución

6.1. Diagnóstico

Al recargar el blog, WordPress mostraba “Error establishing a database connection”. Lo primero que pensamos fue que el problema era la zona de disponibilidad: el servidor web está en us-east-1a y RDS en us-east-1b. Pero no es eso. Dentro de una VPC todas las subredes se comunican mediante la ruta local 10.0.0.0/16, sea cual sea su zona. Además, desde ese mismo servidor el cliente mysql conecta con RDS sin problemas, como se ve en el apartado 4.

La diferencia entre esas pruebas y WordPress es el --ssl. En RDS para MariaDB 11.8 el parámetro require_secure_transport viene activado por defecto, así que el servidor rechaza cualquier conexión sin cifrar. El cliente cifra porque se lo pedimos con --ssl, pero WordPress se conecta sin TLS si no se le indica lo contrario.

6.2. Activar TLS en WordPress

WordPress permite pasar opciones a la conexión con la constante MYSQL_CLIENT_FLAGS. La añadimos a wp-config.php justo encima de la línea /* That's all, stop editing! */:

```
define( 'MYSQL_CLIENT_FLAGS', MYSQLI_CLIENT_SSL |
MYSQLI_CLIENT_SSL_DONT_VERIFY_SERVER_CERT );
```

MYSQLI_CLIENT_SSL hace que la conexión vaya cifrada. MYSQLI_CLIENT_SSL_DONT_VERIFY_SERVER_CERT evita que PHP compruebe el certificado de RDS, porque la autoridad que lo firma no está entre las que trae Amazon Linux; el tráfico va cifrado igualmente. En producción lo correcto sería descargar el certificado raíz de RDS y verificarlo. La alternativa, desactivar require_secure_transport con un grupo de parámetros propio, obliga a reiniciar RDS y deja las conexiones sin cifrar, así que la descartamos.

En la captura se ve la línea ya añadida, entre el comentario de “Add any custom values” y el de “That's all, stop editing!”.

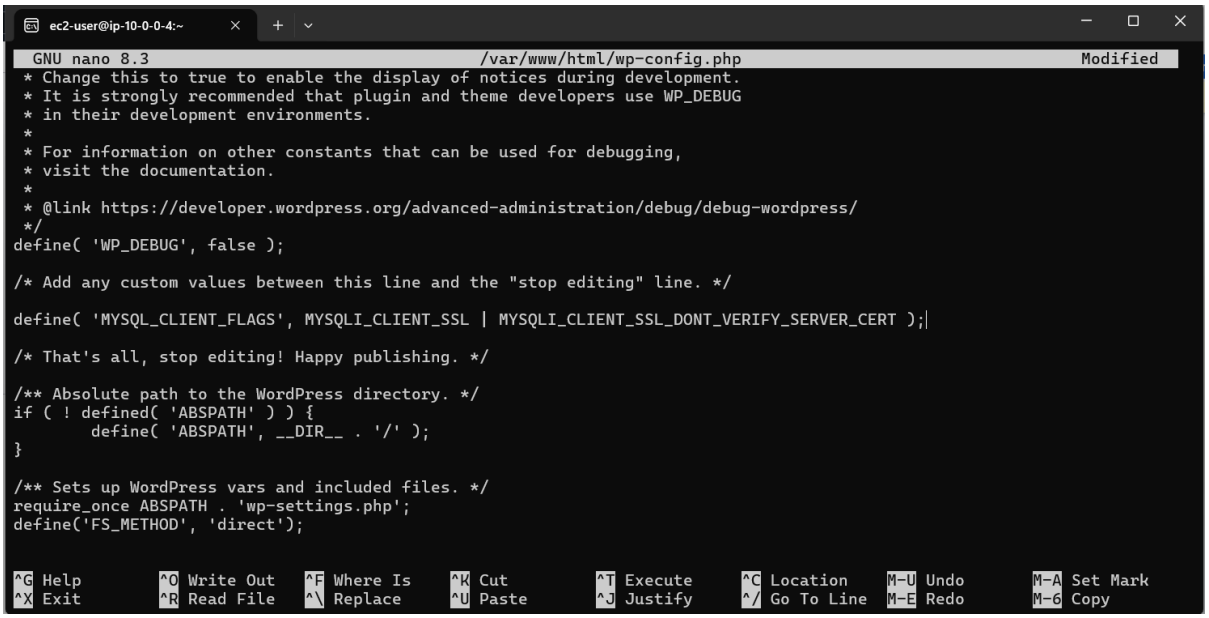


Figura 89. Línea MYSQL_CLIENT_FLAGS añadida en wp-config.php.

6.3. WordPress funcionando sobre RDS

Al recargar la página ya no sale el error. Como wordpressdb estaba vacía, porque no migramos los datos del servidor privado, apareció el asistente de instalación de WordPress. Rellenamos el título del sitio, el usuario y la contraseña, y WordPress creó sus tablas directamente en RDS. En la captura se ve el blog ya instalado, con el título “Servidor Web y WordPress con base de datos RDS”, la entrada de ejemplo y la barra de administración arriba porque hemos iniciado sesión.

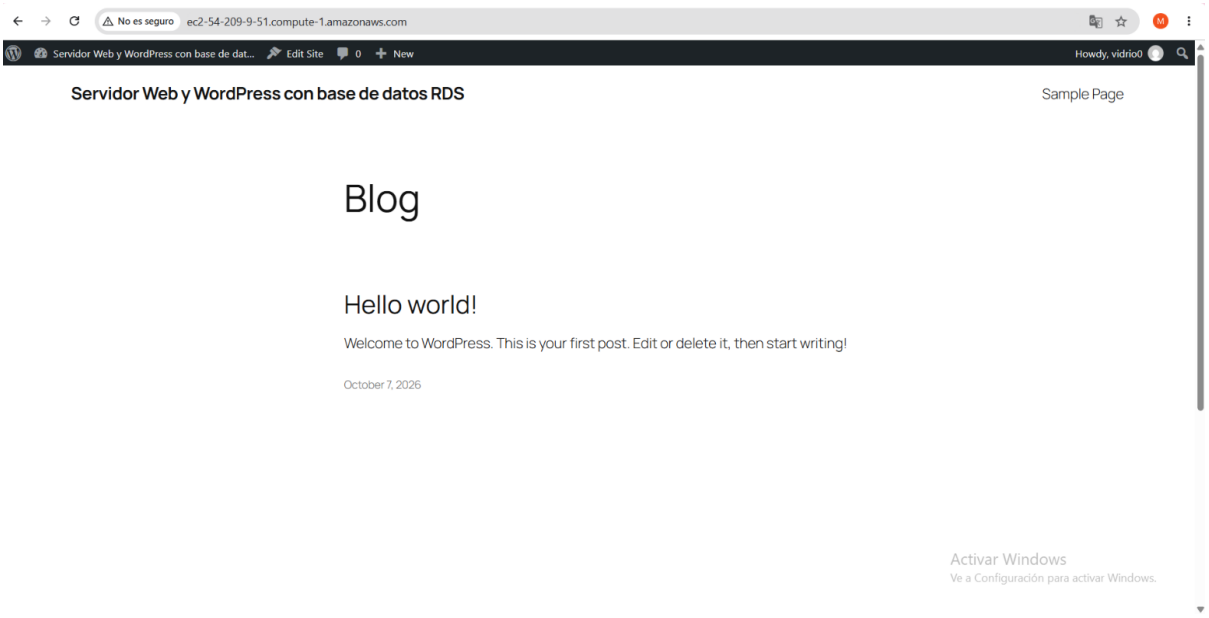


Figura 90. Blog funcionando con la base de datos en RDS.

Para comprobar que los datos están en RDS nos conectamos desde el servidor web con wordpress-user directamente a wordpressdb. SHOW DATABASES sigue mostrando solo information_schema y wordpressdb, y SHOW TABLES lista las 12 tablas wp_ que ha creado WordPress durante la instalación.

```
mysql -h $RDS -u wordpress-user -p --ssl wordpressdb
SHOW DATABASES;
SHOW TABLES;
```

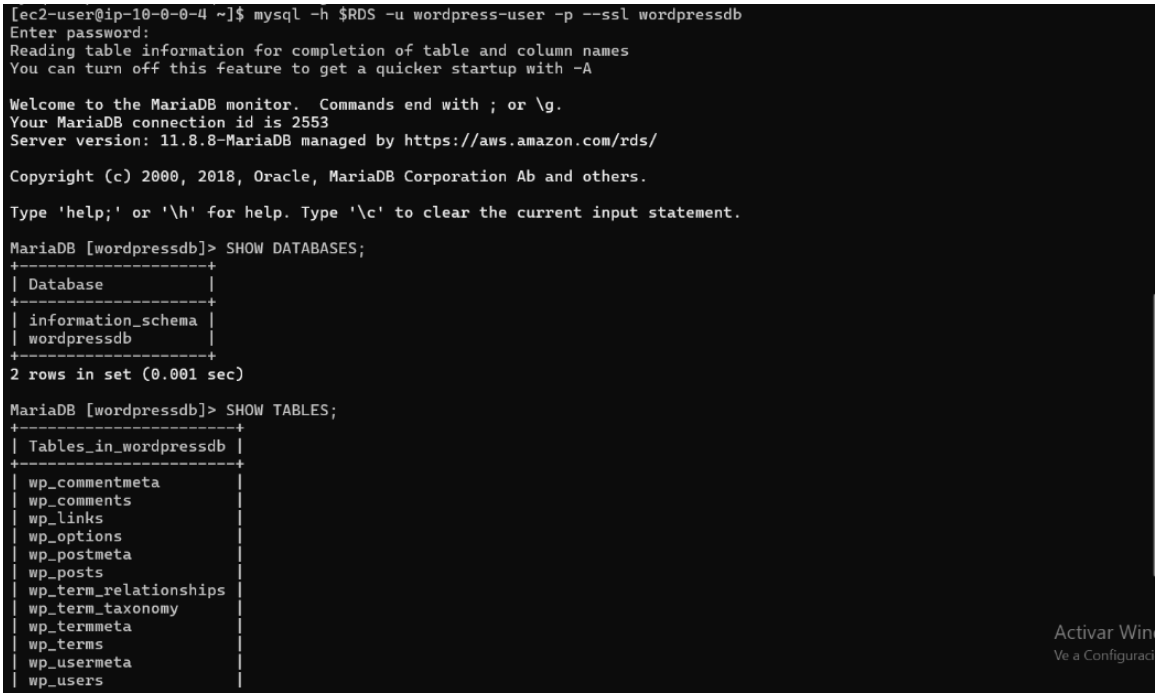


Figura 91. Tablas de WordPress creadas en wordpressdb.

En la tabla wp_users aparece el usuario que creamos en el asistente, vidrio0, registrado el 7 de octubre de 2026 y con la dirección del servidor web como URL. Es decir, lo que escribimos en el formulario de WordPress se ha guardado en RDS.

```
SELECT * FROM wp_users;
```



Figura 92. Usuario de WordPress guardado en la tabla wp_users de RDS.

6.4. El servidor privado ya no hace falta

Como última prueba dejamos parado servidor-privado, que era donde estaba MariaDB en el nivel medio. Filtrando las instancias por estado “running” solo aparece servidor-web-publico.

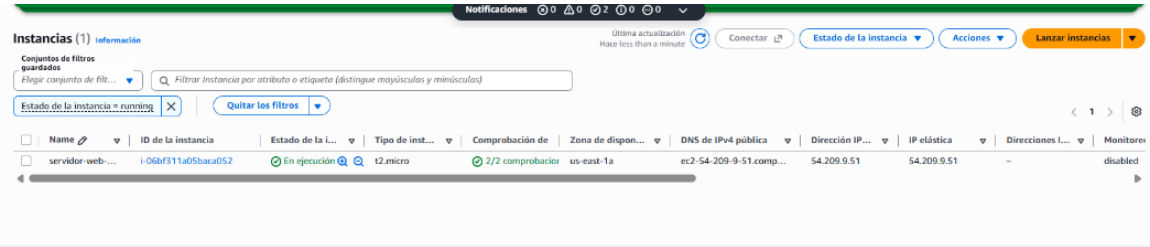


Figura 93. Solo servidor-web-publico en ejecución; servidor-privado parado.

Desde el servidor web ni siquiera podemos entrar por SSH al privado: la conexión a la 10.0.128.5 devuelve “No route to host” porque la instancia está apagada. Aun así, el blog sigue funcionando con normalidad, así que WordPress ya no depende para nada del servidor privado y toda su información está en RDS.

```
ssh -i keypair-cdist.pem ec2-user@10.0.128.5
```



Figura 94. El servidor privado no responde porque está apagado.

7. Comparación con el nivel medio

	Nivel medio	**Nivel avanzado (RDS)**
Base de datos	MariaDB 10.5 en servidor-privado	Amazon RDS for MariaDB 11.8
Quién la mantiene	Nosotros: dnf, systemctl, copias	AWS
Ubicación	Subred privada 10.0.128.0/20 (us-east-1a)	Subred privada 10.0.144.0/20 (us-east-1b)
DB_HOST	10.0.128.5	Endpoint de wordpress-rds ($RDS)
Acceso desde fuera	Ninguno: sin IP pública	Ninguno: sin acceso público
Cifrado de la conexión	No	Obligatorio (TLS)

En resumen, para WordPress solo cambian unas líneas de wp-config.php, pero ya no tenemos que mantener nosotros una máquina para la base de datos y además la conexión con RDS va cifrada.

Anexo: https://github.com/100472851/cloud-entregable-aws